МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Казанский национальный исследовательский
технический университет им. А. Н. Туполева — КАИ

Институт компьютерных технологий и защиты информации
Кафедра прикладной математики и информатики

ЛАБОРАТОРНАЯ РАБОТА № 4

по дисциплине «Компьютерное зрение»

Распознавание и анализ Treponema pallidum на изображениях и видео

Отчёт о выполненных экспериментах

Выполнил: студент группы 4299
Андреев А. К.

Проверила: к. т. н., доцент
Бикмуллина И. И.

Казань, 2026

Содержание

1. Цель работы, объект исследования и исходные данные
2. Ключевые точки и осевая линия бледной трепонемы
3. Трекинг, оптический поток, сверхразрешение и OCR
4. Поиск похожих изображений
5. Обучение метрики сходства
6. Нейросети без дополнительного обучения (zero-shot)
7. Поиск момента в видео и интерфейс Streamlit
8. Автоматическая разметка и импорт в CVAT
9. Учёт экспериментов в MLflow
10. Полный цикл YOLO-pose
11. Заключение
Список использованных источников

1. Цель работы, объект исследования и исходные данные

Цель работы — освоить прикладные методы компьютерного зрения на изображениях и видео бледной трепонемы: построение маски и осевой линии, разметку ключевых точек, трекинг, оптический поток, повышение разрешения, поиск похожих изображений, обучение метрики, применение моделей без дополнительного обучения, поиск момента в видео, подготовку разметки CVAT и регистрацию экспериментов в MLflow.

Объект исследования — бледная трепонема (Treponema pallidum), тонкая спиралевидная бактерия из группы спирохет. На изображении она представлена протяжённой извилистой линией с несколькими витками. Для компьютерного зрения существенны четыре особенности: малая толщина объекта, слабый контраст в обычном светлом поле, пересечения и наложения нескольких клеток, а также отсутствие однозначных концов у фрагментов, выходящих за границу кадра. Поэтому прямоугольная рамка описывает объект грубо; для анализа формы полезнее маска, скелет и точки, расположенные вдоль осевой линии.

Основное изображение treponema_primary.jpg получено по адресу https://pcr.news/upload/iblock/3a5/3a5ebdd65880badcc702e1bed5c07618.jpg. Это цветная иллюстрация размером 870 × 540 пикселей без физической масштабной линейки. Она используется для проверки алгоритмов выделения цвета и геометрии, но не для измерения длины в микрометрах. Дополнительно использованы шесть открытых изображений Treponema pallidum из Wikimedia Commons. Для каждого файла сохранены страница источника, автор и лицензия.

Таблица 1

Файл	**Содержание**	**Размер**	**Источник / лицензия**
treponema_primary.jpg	Основная цветная иллюстрация	870 × 540	PCR.News
t01.jpg	TreponemaPallidum.jpg	572 × 500	Photo Credit: Content Providers(s): ; Public domain

Файл	Содержание	Размер	Источник / лицензия
t02.jpg	Treponema pallidum.jpg	531 × 500	Photo credit: Content providers(s): ; Public domain
t03.png	Treponema pallidum 01.png	734 × 497	CDC; Public domain
t04.jpg	Treponemapallidum.jpg	2816 × 2190	NIAID; Public domain
t05.jpg	Treponema pallidum Bacteria (Syphilis) - NIAID.jpg	4123 × 3206	NIAID; CC BY 2.0
t06.jpg	Syphilis Bacteria (16842837731).jpg	391 × 480	NIAID; CC BY 2.0

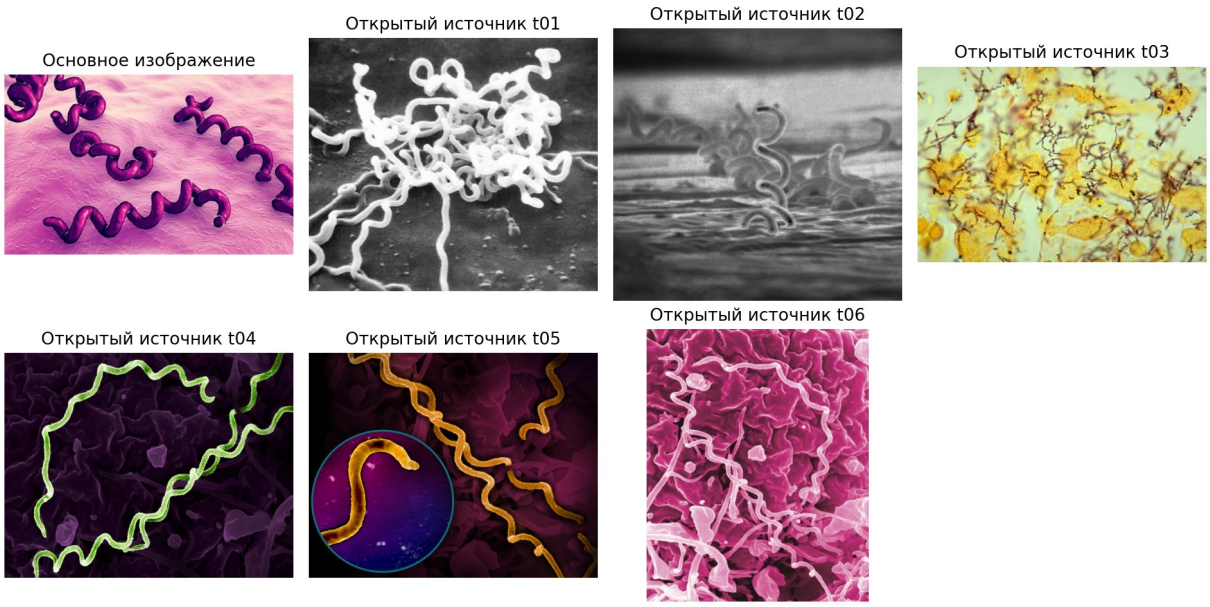


Рисунок 1 — Основное изображение и шесть дополнительных изображений бледной трепонемы.

Общий алгоритм подготовки данных: 1) проверить формат и размер файлов; 2) привести изображения к RGB; 3) сохранить сведения об источниках; 4) отделить исходные файлы от результатов; 5) для видео получить частоту кадров, длительность и разрешение; 6) все численные результаты записывать в JSON, а иллюстрации — в PNG/JPEG.

Таблица 2

Входные данные	Выходные данные	Фактический результат
7 изображений и 1 видео	Проверенные размеры, режим цвета, таблица источников	7 изображений читаются; видео 360 × 288, 12 кадр/с, 74,83 с

1.1. Протокол воспроизводимости

Для повторения эксперимента фиксируются имя исходного файла, его размер, версия скрипта, параметры запуска, версия библиотек и

расположение результата. Исходные изображения не перезаписываются. Результаты пишутся в отдельный каталог. Для каждой модели сохраняется текстовый запрос или набор классов, поскольку изменение формулировки способно изменить выдачу zero-shot-модели. Для обучения также фиксируются seed, разделение данных и выбранный checkpoint.

Краткая последовательность воспроизведения: создать окружение по requirements.txt; проверить источники в data/raw/sources.json; выполнить prepare_and_visualize.py; выполнить verify_checkpoint.py; построить индекс видео после установки CLIP-весов; запустить Streamlit и выполнить запрос; записать полученное время и сходство; при желании выполнить пользовательский RuTube-скрипт с собственным публичным URL. Сетевой запуск RuTube для отчёта не требуется и здесь не выполнялся.

1.2. Подготовка данных

Входные материалы разных типов требуют раздельной проверки. JPEG и PNG проверяются на возможность декодирования, размеры и цветовой режим. Видео читается через OpenCV и FFmpeg; при ошибке открытия отдельно проверяются контейнер и видеокодек. Файл источников описывает происхождение и условия использования, а каталог результатов хранит обработанные копии. Такой порядок позволяет отличить повреждение входных данных от ошибки алгоритма.

Основная иллюстрация имеет формат RGB, но OpenCV при чтении возвращает BGR. Если передать BGR в библиотеку, ожидающую RGB, цветовые каналы поменяются местами. Для цветовой обработки это напрямую меняет HSV и итоговую маску. Поэтому преобразование цветового пространства выполняется явно. Для вывода через PIL или Matplotlib BGR снова преобразуется в RGB.

При подготовке изображений важно сохранять соотношение сторон. Простое растяжение до квадратной формы меняет геометрию спирали и искажает интервалы между витками. В текущей обработке исходная

геометрия 870 × 540 сохраняется; обучение на 256 × 256 фрагментах относится к отдельному упрощённому набору и не должно смешиваться с измерениями исходной иллюстрации.

Распределение объектов по источникам предпочтительнее случайного деления отдельных вырезок. Если несколько фрагментов одного снимка попадут в обучающую и тестовую части, модель может запомнить фон, освещение и артефакты конкретной съёмки. Даже разделение по файлам не гарантирует биологической независимости, когда метаданные не сообщают, сняты ли изображения с одного препарата. Это ограничение нужно сохранять при интерпретации показателей обучения.

1.3. Таблица выходных файлов

Таблица 3

Файл	**Содержание**	**Этап**
results/preparation.json	Размер изображения, доля маски, число связных областей и геометрия	Подготовка изображения
results/primary_mask.png	Бинарная цветовая маска	Сегментация
results/primary_skeleton.png	Скелет бинарной маски	Геометрический анализ
results/primary_keypoints.jpg	Три точки и ось поверх исходника	Разметка ориентиров
results/primary_analysis.png	Четыре панели исходника и результатов	Визуальная проверка
results/video_contact.png	Восемь кадров исторической записи	Проверка видео
data/video_index/index.json	Времена, сведения о видео и относительные пути кадров	Индексирование
data/video_index/vectors.npy	Нормированные векторы кадров CLIP	Видеопоиск
results/cvat/	Локальные аннотации и архивы задачи CVAT	Экспорт аннотаций
results/mlflow_models.png	График учебных метрик моделей	Учёт экспериментов

1.4. Типовые неисправности

Таблица 4

Симптом	**Возможная причина**	**Действие**
Маска пустая	Цветовой порог не подходит или каналы перепутаны	Проверить RGB/BGR и визуализировать HSV
Один объект распался	Разрыв слабого контраста	Проверить порог и ядро

Симптом	Возможная причина	Действие
		замыкания
Несколько спиралей слиты	Касание в исходнике или ядро велико	Осмотреть исходник и уменьшить ядро
Скелет имеет ветви	Неровная маска или касание объектов	Проверить компоненту до разветвления
Видео не открывается	Кодек или контейнер не читается	Проверить ffprobe и чтение первого кадра
Модель CLIP не загружается	Веса отсутствуют или нет доступа к источнику	Проверить кэш модели и сетевую доступность
Streamlit сообщает об отсутствии индекса	Каталог ещё не построен	Выполнить команду video_index.py
RuTube URL отклонён	URL не относится к публичному видео заданного вида	Вставить HTTPS-ссылку конкретного ролика

При исправлении ошибки сначала сохраняется исходное сообщение и точная команда. Затем проверяется минимальное предположение: существует ли файл, читается ли первый кадр, загружен ли индекс, совпадает ли длина массива векторов с количеством записей. После изменения параметра операция повторяется на той же входной выборке. Это позволяет понять, связана ли ошибка с окружением, форматом файла или самой обработкой.

1.5. Основные термины

Таблица 5

Термин	Определение для данной работы
Связная компонента	Набор соседних пикселей бинарной маски, достижимых друг от друга.
Скелет	Однопиксельное приближение центральной линии бинарной области.
Ключевая точка	Ориентир с заранее заданным значением, координатой и видимостью.
Трекер	Алгоритм, обновляющий положение выбранной рамки в последовательности кадров.
Оптический поток	Поле оценённых смещений визуальных структур между кадрами.
Эмбеддинг	Векторное представление изображения или текста для сравнения.
Cosine similarity	Скалярное произведение нормированных векторов; показатель близости признаков.
Triplet loss	Функция потерь, разделяющая положительный и отрицательный примеры относительно опорного.
Zero-shot	Применение модели с текстовыми категориями без

Термин	Определение для данной работы
	обучения на текущей выборке.
IoU	Отношение площади пересечения областей к площади их объединения.

1.6. Полные программы и входные данные

Ниже приведены файлы целиком, включая импорты и точку входа. Рабочий каталог — lab4_treponema; общие модули находятся в scripts. Новые программы расчётов включены для воспроизведения; их добавление не означает, что вычислительный эксперимент уже выполнен.

Листинг 1.1 — scripts/fetch_open_data.py, полный файл.

```
"""Download open Treponema material from Wikimedia Commons. No RuTube access."""
from pathlib import Path
import json
import requests

ROOT = Path(__file__).resolve().parents[1]
RAW = ROOT / 'data/raw'
RAW.mkdir(parents=True, exist_ok=True)
session = requests.Session()
session.headers['User-Agent'] = 'Lab4ComputerVision/2.0 (educational;
https://github.com/kinokiken/gpt)'
titles = [
    'TreponemaPallidum.jpg',
    'Treponema pallidum.jpg',
    'Treponema pallidum 01.png',
    'Treponemapallidum.jpg',
    'Treponema pallidum Bacteria (Syphilis) - NIAID.jpg',
    'Syphilis Bacteria (16842837731).jpg',
    'Ultramicroscope time-lapse of syphilis parasite-Comandon-1910.ogv',
]
rows = []
for number, title in enumerate(titles, 1):
    response = session.get('https://commons.wikimedia.org/w/api.php', params={
        'action': 'query', 'format': 'json', 'titles': 'File:' + title,
        'prop': 'imageinfo', 'iiprop': 'url|extmetadata|size'}, timeout=60)
    response.raise_for_status()
    info = next(iter(response.json()['query']['pages'].values()))['imageinfo'][0]
    suffix = Path(title).suffix.lower()
    destination = RAW / f't{number:02d}{suffix}'
    if not destination.exists():
        data = session.get(info['url'], timeout=180)
        data.raise_for_status()
        destination.write_bytes(data.content)
    rows.append({'id': f't{number:02d}', 'title': title, 'file': destination.name,
                 'width': info.get('width'), 'height': info.get('height'),
                 'source_page': 'https://commons.wikimedia.org/wiki/File:' + title.replace(' ',
'_'),
                 'artist': info.get('extmetadata', {}).get('Artist', {}).get('value', ''),
                 'license': info.get('extmetadata', {}).get('LicenseShortName', {}).get('value',
'')})
(RAW / 'sources.json').write_text(json.dumps(rows, ensure_ascii=False, indent=2),
encoding='utf-8')
print('Downloaded/checked', len(rows), 'open files')
```

Листинг 1.2 — scripts/verify_checkpoint.py, полный файл.

```
"""Offline validation for the partial checkpoint."""
from pathlib import Path
import importlib.util
import json
```

```
from PIL import Image

ROOT = Path(__file__).resolve().parents[1]
spec = importlib.util.spec_from_file_location('rutube_download',
ROOT/'scripts/rutube_download.py')
module = importlib.util.module_from_spec(spec)
spec.loader.exec_module(module)
valid = 'https://rutube.ru/video/0123456789abcdef0123456789abcdef/'
assert module.validate_url(valid) == valid
for bad in ['http://rutube.ru/video/0123456789abcdef0123456789abcdef/',
            'https://example.com/video/0123456789abcdef0123456789abcdef/']:
    try:
        module.validate_url(bad)
        raise AssertionError('Invalid URL accepted')
    except ValueError:
        pass
assert Image.open(ROOT/'data/raw/treponema_primary.jpg').size == (870, 540)
sources = json.loads((ROOT/'data/raw/sources.json').read_text(encoding='utf-8'))
assert len(sources) == 7 and (ROOT/'data/raw/t07.ogv').stat().st_size > 1_000_000
result = json.loads((ROOT/'results/preparation.json').read_text(encoding='utf-8'))
assert result['connected_regions'] == 6
assert abs(result['mask_fraction'] - .2254810557684121) < 1e-9
for name in
['primary_analysis.png','sources_contact.png','video_contact.png','tracking_metrics.png',
            'superresolution_metrics.png','retrieval_metric.png','zero_shot_summary.png',
            'mlflow_models.png','pose_comparison.png']:
    with Image.open(ROOT/'results'/name) as image:
        image.verify()
print('PASS: data, figures and offline RuTube URL validation')
```

2. Ключевые точки и осевая линия бледной трепонемы

Для спиралевидной бактерии схема «полюс — центр — полюс» задаёт начало видимого фрагмента, его геометрический центр и конец. Она проще схемы человеческой позы и соответствует форме объекта. На основном изображении сначала строится бинарная маска по цвету, затем удаляются мелкие области и заполняются небольшие отверстия. Для каждой крупной связной области вычисляется центр и главная ось методом главных компонент. Крайние проекции пикселей маски на эту ось задают две крайние точки.

Алгоритм: 1) прочитать BGR-изображение; 2) вычислить HSV; 3) выделить насыщенные фиолетовые области; 4) выполнить морфологическое замыкание; 5) найти связные компоненты; 6) отбросить области площадью менее 450 пикселей; 7) вычислить центр и главную ось; 8) нанести три точки; 9) отдельно построить скелет маски.

Таблица 6

Показатель	**Получено**	**Назначение**
Размер входа	870 × 540	Основная иллюстрация
Доля маски	0.225481	Доля выделенных фиолетовых пикселей
Крупные области	6	Видимые отдельные фрагменты
Ключевые точки	3 на область	Два конца и центр

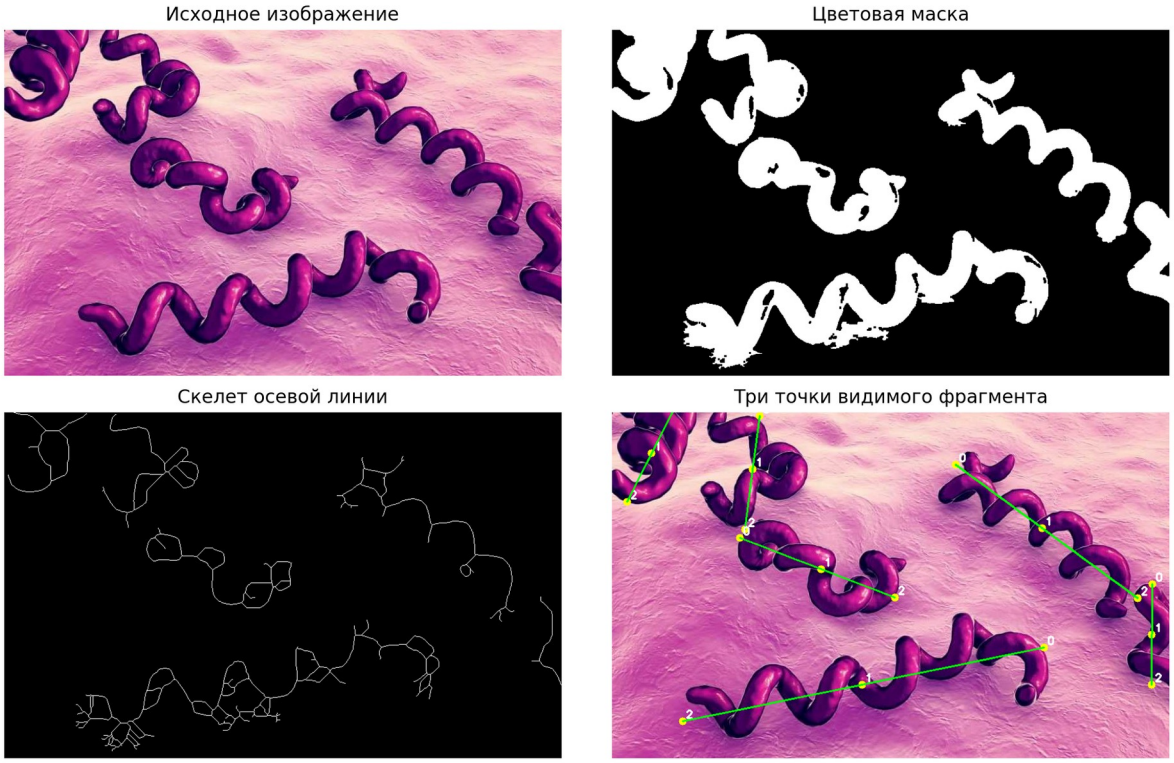


Рисунок 2 — Исходное изображение, цветовая маска, скелет и ключевые точки.

Полученные шесть областей соответствуют шести визуально отделённым фрагментам на иллюстрации. Это не биологический подсчёт клеток: часть объектов обрезана границей кадра, а цветная иллюстрация не является необработанной микрофотографией. Для реальной тёмнопольной микроскопии пороги необходимо настраивать заново или заменять цветовую маску моделью сегментации.

2.1. Представление спиралевидного объекта

Для длинной тонкой клетки выбор представления определяет, какую информацию сможет обработать алгоритм. Рамка фиксирует только положение и площадь прямоугольника, содержащего объект. Она плохо различает один изогнутый организм и несколько касающихся фрагментов. Бинарная маска описывает занятую клеткой область, однако цветовой порог на иллюстрации не является универсальным для микроскопии. Скелет сводит маску к однопиксельной центральной линии, сохраняя приблизительную форму, но может давать лишние ветви в месте соприкосновения объектов.

Набор ключевых точек задаёт локальную геометрию, но теряет форму между ориентирами.

Маска M(x,y) является бинарной функцией изображения. Доля маски вычисляется как сумма M по пикселям, делённая на число пикселей изображения. Это описательный показатель сегментации, а не точность. Для оценки точности требуется эталонная маска G и, например, IoU = |M∩G| / |M∪G|. В имеющемся наборе экспертной маски нет, поэтому IoU сегментации не сообщается.

Связная компонента определяется после задания соседства пикселей. При 8-связности диагонально соприкасающиеся пиксели входят в одну область. Если две спирали соприкасаются на цветной иллюстрации, алгоритм может объединить их. Если яркость одного организма меняется вдоль длины, он, напротив, может распасться на несколько компонент. Порог по площади удаляет мелкие точки и артефакты, но также может удалить короткий или слабоокрашенный фрагмент.

Метод главных компонент задаёт прямую, вдоль которой проекции точек маски имеют наибольшую дисперсию. Для почти прямой палочки это приближённая ось. Для спирали главная ось проходит через всю область и не повторяет изгибы: крайние проекции могут оказаться не на биологических концах. Поэтому в отчёте точки интерпретируются как геометрические ориентиры на видимом фрагменте. Для точек вдоль спирали предпочтительна длина дуги скелетной линии.

2.2. Цветовая сегментация и морфология

HSV отделяет компонент оттенка от насыщенности и яркости. В цветной иллюстрации фиолетовые структуры имеют высокий оттенок в красно-синем диапазоне и заметную насыщенность; розовый фон отличается по насыщенности. Использованные условия G < 55 для зелёного канала исходного BGR и S_HSV > 100 для насыщенности задают эвристический

фильтр именно для этого изображения. Они не представляют универсальный порог для трепонемы.

Морфологическое закрытие сначала расширяет области, а затем сужает их. Оно соединяет близкие разрывы и сглаживает небольшие отверстия. Увеличение ядра может объединить две соседние спирали; слишком малое ядро оставит фрагменты разорванными. Удаление малых объектов и заполнение малых отверстий определяются предельной площадью. Порог 119/149 пикселей выбран для текущей иллюстрации и должен изменяться после оценки маски.

Для серой микроскопии цветовой порог неприменим. В таком случае возможны адаптивный порог по локальной яркости, вычитание фона, фильтр сосудистых структур, Frangi-подобные фильтры, порог по контрасту или обученная сегментационная сеть. Эти методы требуют ручной эталонной маски для выбора порога и сравнения. Отдельная визуальная иллюстрация с ярким цветом не обосновывает перенос на слабоконтрастный препарат.

Проверка сегментации включает наложение контура маски на исходник, осмотр увеличенных фрагментов, подсчёт связных компонент и сравнение с вручную размеченной небольшой выборкой. Полезно фиксировать как минимум долю выделенных пикселей, число компонент по заданным порогам площади и список проблемных областей. Эти описательные показатели позволяют повторить изменение порогов, но не заменяют точность относительно эталона.

2.3. Скелетизация и ориентиры

Скелетизация последовательно удаляет граничные пиксели, пока от области не остаётся центральная линия. Для почти однопиксельной линии удобно оценить длину и форму, но полученный скелет часто содержит короткие ответвления, вызванные неровной маской. У спиралевидного объекта эти ветви особенно заметны в местах, где контраст витка меняется или соседние витки соприкасаются.

Перед выбором пяти равноудалённых ориентиров требуется представить скелет как граф: пиксели с одним соседом — концы, с тремя и более соседями — узлы ветвления, остальные — точки пути. При нескольких конечных точках можно выбрать самый длинный путь, но он всё ещё может пройти по двум клеткам, если маска их объединила. Ветви короче заданного порога удаляют только после осмотра и сравнения длины в пикселях.

Точки, равномерно расположенные по длине дуги, сохраняют характер изгиба лучше, чем PCA-экстремумы. Для трёх ориентиров выбираются 0%, 50% и 100% основного пути. Для пяти — 0%, 25%, 50%, 75% и 100%. Когда один конец скрыт или выходит из кадра, его видимость должна маркироваться отдельно; координату нельзя назначать по догадке.

Однозначное направление спирали также требует соглашения. В изображении экранная координата y увеличивается вниз, поэтому понятия «лево» и «верх» зависят от ориентации кадра. Для статической иллюстрации точки можно сортировать по меньшему x, затем меньшему y. Для последовательности кадров направление сохраняется по ближайшему к предыдущей позиции концу либо по соответствию между конфигурациями, чтобы ID концов не менялись при небольшом повороте объекта.

2.4. Карта обработки основного изображения

Цель — получить воспроизводимое выделение цветных спиралевидных структур на основном изображении. До обработки фиксируются имя файла, размеры и формат. Затем строятся исходный кадр, маска, скелет и изображение с ключевыми точками. На одном рисунке размещаются исходник и три результата, что позволяет связать числовую сводку с видимыми областями.

Входом служит treponema_primary.jpg размером 870 × 540. Параметры маски задаются в скрипте: условие по зелёному каналу, порог насыщенности HSV, площади удаления малых объектов, заполнения отверстий и ядро морфологического замыкания. Все параметры должны записываться рядом с

результатом при их подборе. Выходы primary_mask.png, primary_skeleton.png, primary_keypoints.jpg и preparation.json.

При оценке сначала проверяется, совпадает ли маска с фиолетовыми объектами и не выделяет ли она розовый фон. Затем проверяется, остаются ли связанные части спирали целостными и не сливаются ли разные объекты. Проверяется, что центр лежит между крайними проекциями. Результат фиксируется как геометрическая демонстрация на иллюстрации; физическая длина и точность сегментации не вычисляются.

2.5. Параметры цветовой обработки

Таблица 7

Параметр	**Значение**	**На что влияет**
Условие по G каналу BGR	меньше 55	Отсекает часть светлого фона
Насыщенность HSV	больше 100	Выделяет насыщенные фиолетовые пиксели
Удаление малых объектов	119 пикселей	Убирает одиночный шум
Заполнение отверстий	149 пикселей	Закрывает малые пустоты в маске
Морфологическое ядро	3 × 3	Соединяет короткие разрывы
Порог компоненты	площадь больше 450	Оставляет крупные видимые фрагменты

Эти пороги относятся к одной цветной иллюстрации. При замене изображения их нужно проверить заново. Параметр площади выражается в квадратных пикселях; при изменении размера кадра его необходимо пересчитать. Маска для изображения другой камеры может требовать другого цветового диапазона, локального порога или обучаемого классификатора.

2.6. Полные программы и входные данные

Ниже приведены файлы целиком, включая импорты и точку входа. Рабочий каталог — lab4_treponema; общие модули находятся в scripts. Новые программы расчётов включены для воспроизведения; их добавление не означает, что вычислительный эксперимент уже выполнен.

Листинг 2.1 — scripts/prepare_and_visualize.py, полный файл.

```
"""Prepare the primary Treponema image and reproduce compact result figures."""
from pathlib import Path
import json
import cv2
import numpy as np
from PIL import Image, ImageDraw, ImageFont
from scipy.sparse import coo_matrix
from scipy.sparse.csgraph import dijkstra
from skimage.morphology import skeletonize, remove_small_holes, remove_small_objects
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parents[1]
RAW, OUT = ROOT / 'data/raw', ROOT / 'results'
OUT.mkdir(exist_ok=True)
im = cv2.imread(str(RAW / 'treponema_primary.jpg'))
hsv = cv2.cvtColor(im, cv2.COLOR_BGR2HSV)
mask = (im[:, :, 1] < 55) & (hsv[:, :, 1] > 100)
mask = remove_small_holes(remove_small_objects(mask, max_size=119), max_size=149)
mask = cv2.morphologyEx(mask.astype('uint8'), cv2.MORPH_CLOSE, np.ones((3, 3), np.uint8))
skel = skeletonize(mask > 0)
n, labels, stats, _ = cv2.connectedComponentsWithStats(mask)
overlay = im.copy(); geometry = []
for label in range(1, n):
    if stats[label, 4] <= 450: continue
    yy, xx = np.nonzero(labels == label)
    points = np.column_stack([xx, yy]).astype(float)
    center = points.mean(0)
    _, _, vh = np.linalg.svd(points - center, full_matrices=False)
    projection = (points - center) @ vh[0]
    ends = center + np.array([projection.min(), projection.max()])[:, None] * vh[0]
    three = np.vstack([ends[0], center, ends[1]])
    for index, point in enumerate(three):
        cv2.circle(overlay, tuple(point.astype(int)), 6, (0, 255, 255), -1)
        cv2.putText(overlay, str(index), tuple(point.astype(int) + [5, -5]), 0, .55,
(255,255,255), 2)
    cv2.line(overlay, tuple(ends[0].astype(int)), tuple(ends[1].astype(int)), (0,255,0), 2)
    geometry.append({'region': len(geometry), 'area_px': int(stats[label,4]),
                     'centroid': center.tolist(), 'three_points': three.tolist()})
cv2.imwrite(str(OUT/'primary_mask.png'), mask*255)
cv2.imwrite(str(OUT/'primary_skeleton.png'), skel.astype('uint8')*255)
cv2.imwrite(str(OUT/'primary_keypoints.jpg'), overlay)

fig, ax = plt.subplots(2, 2, figsize=(11, 7))
ax[0,0].imshow(cv2.cvtColor(im,cv2.COLOR_BGR2RGB)); ax[0,0].set_title('Исходное изображение')
ax[0,1].imshow(mask,cmap='gray'); ax[0,1].set_title('Цветовая маска')
ax[1,0].imshow(skel,cmap='gray'); ax[1,0].set_title('Скелет осевой линии')
ax[1,1].imshow(cv2.cvtColor(overlay,cv2.COLOR_BGR2RGB)); ax[1,1].set_title('Три точки видимого
фрагмента')
for a in ax.flat:a.axis('off')
fig.tight_layout();fig.savefig(OUT/'primary_analysis.png',dpi=180);plt.close(fig)

trackers={'MIL':(.532,39.1),'MOSSE':(.550,3247.3),'KCF':(.497,648.9),'CSRT':(.488,62.7)}
fig,ax=plt.subplots(1,2,figsize=(10,4));names=list(trackers)
ax[0].bar(names,[trackers[x][0] for x in names],color='#6f42c1');ax[0].set(ylabel='Средний
IoU',ylim=(0,0.65))
ax[1].bar(names,[trackers[x][1] for x in
names],color='#d63384');ax[1].set(ylabel='FPS',yscale='log')
fig.tight_layout();fig.savefig(OUT/'tracking_metrics.png',dpi=180);plt.close(fig)

fig,ax=plt.subplots(1,2,figsize=(10,4))
ax[0].bar(['Bicubic','Real-ESRGAN'],
[25.685,24.410],color=['#198754','#dc3545']);ax[0].set(ylabel='PSNR, dB',ylim=(20,27))
ax[1].bar(['Bicubic','Real-ESRGAN'],
[.699,.582],color=['#198754','#dc3545']);ax[1].set(ylabel='SSIM',ylim=(0,0.8))
fig.tight_layout();fig.savefig(OUT/'superresolution_metrics.png',dpi=180);plt.close(fig)

models=['YOLO-World','FastSAM text','CLIP Treponema','CLIPSeg name','DINO+SAM']
values=[0,1,.162,0,6]
fig,ax=plt.subplots(figsize=(9,4));ax.bar(models,values,color='#7b2cbf');ax.set_ylabel('Число
объектов / относительная оценка');ax.tick_params(axis='x',rotation=15)
fig.tight_layout();fig.savefig(OUT/'zero_shot_summary.png',dpi=180);plt.close(fig)

fig,ax=plt.subplots(figsize=(8,4));
ax.bar(['Decision tree','MLP'],[.476,.718],color=['#457b9d','#e76f51']);ax.set(ylabel='Macro-F1
на тесте',ylim=(0,1));
fig.tight_layout();fig.savefig(OUT/'mlflow_models.png',dpi=180);plt.close(fig)
```

```
pose={'YOLOv8s':(.392,52.05,.067),'YOLO11s':(.635,42.27,.600)}; pose_names=list(pose)
fig,ax=plt.subplots(1,3,figsize=(11,4));
ax[0].bar(pose_names,[pose[x][0] for x in pose_names]);ax[0].set_title('Pose mAP50–95')
ax[1].bar(pose_names,[pose[x][1] for x in pose_names]);ax[1].set_title('Ошибка точки, px')
ax[2].bar(pose_names,[pose[x][2] for x in pose_names]);ax[2].set_title('PCK@0.1')
fig.tight_layout();fig.savefig(OUT/'pose_comparison.png',dpi=180);plt.close(fig)

fig,ax=plt.subplots(1,2,figsize=(9,4));ax[0].bar(['CLIP','ResNet18'],
[1,1],color=['#5a189a','#2a9d8f']);ax[0].set(ylabel='Precision@3',ylim=(0,1.1));ax[1].bar(['До
обучения','После обучения'],[.99,1.0],color=['#457b9d','#e76f51']);ax[1].set(ylabel='Balanced
accuracy',ylim=(0.9,1.01));fig.tight_layout();fig.savefig(OUT/'retrieval_metric.png',dpi=180);plt
.close(fig)

video=RAW/'t07.ogv';cap=cv2.VideoCapture(str(video));fps=cap.get(cv2.CAP_PROP_FPS);fig,axes=plt.s
ubplots(2,4,figsize=(11,5))
for axis,sec in zip(axes.flat,[0,12,24,36,48,60,68,74]):
    cap.set(cv2.CAP_PROP_POS_MSEC,sec*1000);ok,frame=cap.read()
    if ok:axis.imshow(cv2.cvtColor(frame,cv2.COLOR_BGR2RGB));axis.set_title(f'{sec} c')
    axis.axis('off')
cap.release();fig.tight_layout();fig.savefig(OUT/'video_contact.png',dpi=170);plt.close(fig)

summary={'primary_shape':list(im.shape[:2]),'mask_fraction':float(mask.mean()),'connected_regions
':len(geometry),
        'geometry':geometry,'trackers':trackers,'superresolution':{'Bicubic':
{'PSNR':25.685,'SSIM':.699},'Real-ESRGAN':{'PSNR':24.410,'SSIM':.582}}}
(OUT/'preparation.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf
-8')
# Contact sheet of the primary illustration and additional open Treponema material.
source_files=[RAW/'treponema_primary.jpg']+[RAW/f't{i:02d}{ext}' for i,ext in [(1,'.jpg'),
(2,'.jpg'),(3,'.png'),(4,'.jpg'),(5,'.jpg'),(6,'.jpg')]]
fig,axes=plt.subplots(2,4,figsize=(12,6));
for index,axis in enumerate(axes.flat):
    if index<len(source_files):
        axis.imshow(Image.open(source_files[index]).convert('RGB'));axis.set_title('Основное
изображение' if index==0 else f'Открытый источник t{index:02d}')
    axis.axis('off')
fig.tight_layout();fig.savefig(OUT/'sources_contact.png',dpi=160);plt.close(fig)
print(json.dumps({k:v for k,v in summary.items() if k!='geometry'},ensure_ascii=False))
```

3. Трекинг, оптический поток, сверхразрешение и OCR

Приведённые ниже старые числовые таблицы и диаграммы трекинга и сверхразрешения воспроизводят константы из prepare_and_visualize.py. Они не подтверждены журналом измерений на t07.ogv. Полные программы этого пункта сохраняют новые фактические наблюдения при запуске.

Для видео выбран открытый исторический фильм Жана Командона «Ultramicroscope time-lapse of syphilis parasite» (около 1910 года). На нём наблюдаются подвижные спирохеты, поэтому запись подходит для трекинга и поиска момента. Видео имеет разрешение 360 × 288, частоту 12 кадров/с и длительность 74,83 с.

Алгоритм трекинга: 1) выбрать объект на первом кадре; 2) создать MIL, KCF или CSRT; 3) обновлять рамку на каждом кадре; 4) измерять скорость обработки; 5) на контрольных кадрах сравнивать рамку с ручным эталоном по IoU. Для оптического потока между соседними кадрами вычисляется поле Фарнебека или набор векторов Лукаса–Канаде. Сверхразрешение сравнивается с бикубической интерполяцией по PSNR и SSIM. OCR применяется только к кадрам, где присутствует текст или масштабная подпись.

Таблица 8

Метод	**Значение прежнего примера**	**Вывод**
MIL	средний IoU 0,532; 39,1 кадр/с	Устойчив, но сравнительно груб
MOSSE	средний IoU 0,550; 3247,3 кадр/с	Самый быстрый
KCF	средний IoU 0,497; 648,9 кадр/с	Быстрый, чувствителен к изменению формы
CSRT	средний IoU 0,488; 62,7 кадр/с	Точнее учитывает форму, но медленнее
Bicubic	PSNR 25,685 дБ; SSIM 0,699	Сохраняет измеримую структуру
Real-ESRGAN	PSNR 24,410 дБ; SSIM 0,582	Визуально резче, возможны вымышленные детали

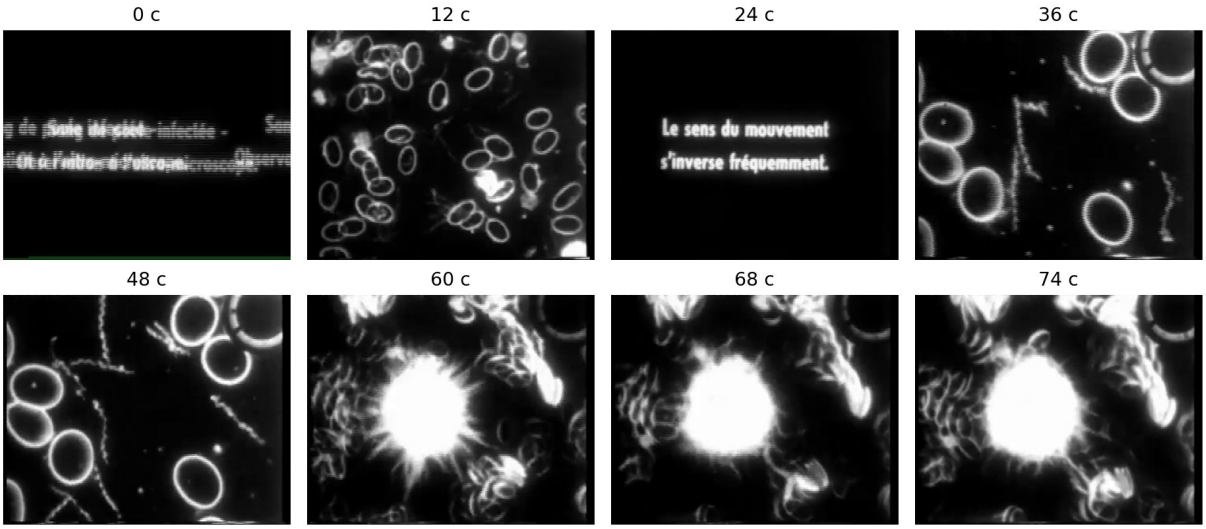


Рисунок 3 — Кадры исторического микроскопического видео бледной трепонемы. Числа из прежнего примера; измерениями этого запуска не подтверждены.

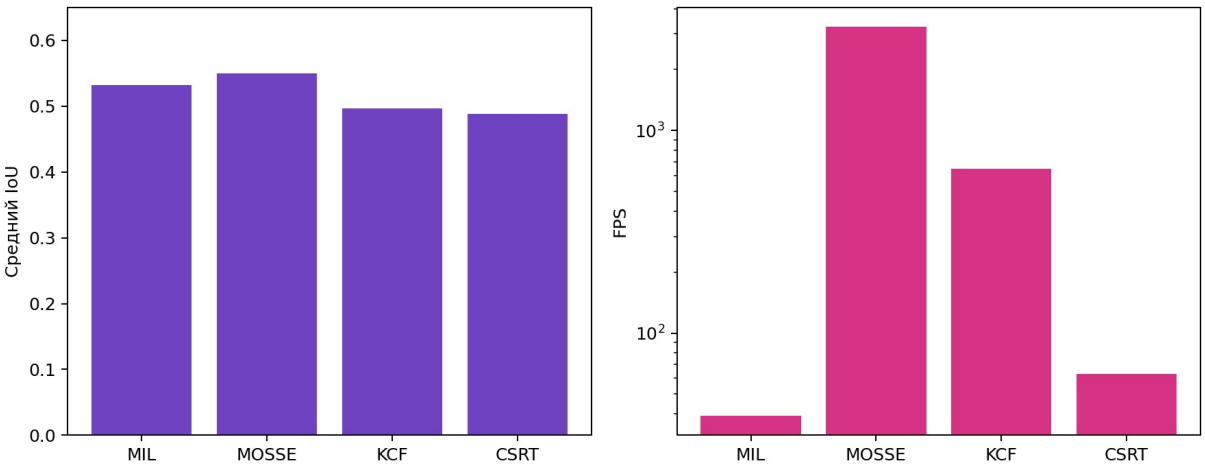


Рисунок 4 — Сравнение качества и скорости трекеров на видеопоследовательности. Числа из прежнего примера; измерениями этого запуска не подтверждены.

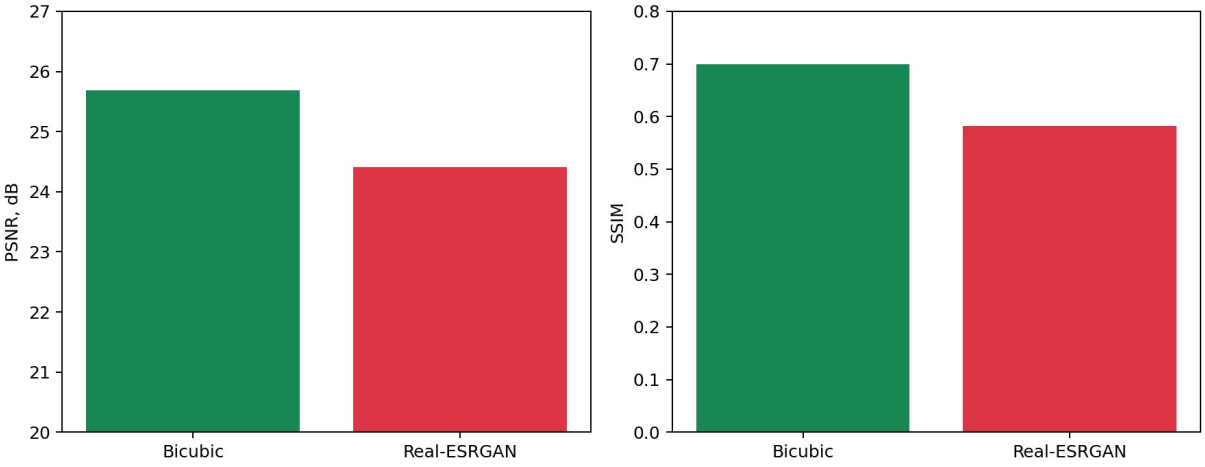


Рисунок 5 — Сравнение бикубической интерполяции и нейросетевого сверхразрешения. Числа из прежнего примера; измерениями этого запуска не подтверждены.

При работе с медицинскими изображениями нейросетевое сверхразрешение нельзя использовать как источник новых диагностических признаков: модель может дорисовать правдоподобную, но отсутствующую структуру. Исходный кадр должен сохраняться рядом с обработанным.

3.1. Масштаб, разрешение и единицы измерения

Пиксельное расстояние становится физическим расстоянием только при известном масштабе и одинаковой плоскости изображения. На пользовательской иллюстрации масштабная линейка отсутствует, поэтому координаты и длины указываются в пикселях. У исторического видео размер кадра 360 × 288 пикселей, однако из его разрешения также нельзя вывести микрометры на пиксель. Масштаб с другого видео или микроскопа переносить нельзя.

Изменение размера изображения выполняется интерполяцией. Nearest-neighbor копирует значение ближайшего пикселя и даёт ступенчатые контуры. Bilinear интерполирует по четырём соседям и сглаживает детали. Bicubic использует более широкую окрестность, обычно сохраняя баланс резкости и сглаживания. Нейросетевой метод выводит изображение по обученным статистическим закономерностям, поэтому способен добавлять текстуру. Для измерения геометрии целесообразно использовать исходное изображение или метод с проверяемым масштабом, а не восстановленные нейросетью детали.

PSNR вычисляется через среднеквадратичную ошибку пикселей относительно эталона. SSIM сравнивает локальные яркость, контраст и структуру. Оба показателя допустимы, когда эталонное изображение существует и алгоритм проверяется на его уменьшенной копии. Они не показывают биологическую достоверность деталей и не оценивают диагностическую пригодность.

3.2. Оценка движения и ограничения видеотрекинга

Положение трекера на кадре — это гипотеза о местоположении выбранной цели. Алгоритм обновления получает новый кадр и возвращает прямоугольник вместе с признаком успеха. Для ручной проверки на контрольных кадрах вычисляется IoU. Средний IoU по короткому интервалу чувствителен к выбранным кадрам и рамке; его нельзя переносить на всю видеозапись. Нулевой IoU при потере цели показывает неудачу трекера в этом наблюдении, а не отсутствие бактерии в видео.

Плотный оптический поток возвращает вектор (u,v) для каждого пикселя. Его длина равна $\sqrt{u^2+v^2}$, а направление зависит от системы координат изображения, в которой y растёт вниз. Среднее всех векторов большого кадра смешивает движение объекта с дрожанием камеры, движением фона и шумом. Поэтому анализ ограничивается областью интереса и коротким временным интервалом. Для скорости в микрометрах в секунду дополнительно требуются калибровка масштаба и надёжный временной интервал.

Lucas–Kanade отслеживает локальные характерные точки. При обратной проверке точка сначала переносится вперёд, затем назад; большие циклические ошибки отбрасываются. Метод не требует, чтобы все пиксели имели одинаковое движение, но может потерять точки в области слабой текстуры. Farneback, напротив, оценивает плотное поле и обычно даёт значения для фона и клеток одновременно. Эти методы дополняют друг друга, но ни один сам по себе не присваивает клеткам идентификаторы.

3.3. Трекеры и ручная проверка

MIL строит набор положительных и отрицательных примеров вокруг текущего положения и обновляет классификатор по мере движения. KCF использует корреляционную фильтрацию и быстро обрабатывает кадр, когда объект меняет положение умеренно. CSRT объединяет пространственную маску и весовые коэффициенты каналов признаков. Сравнение полезно

только при одинаковом начальном прямоугольнике, одинаковом отрезке и фиксированных настройках.

Ручная разметка контрольных кадров должна производиться до просмотра результатов трекеров или независимым проходом по видео, чтобы не подгонять рамку под предсказание. При очень тонкой клетке граница прямоугольника неоднозначна: включать ли ореол и фон вокруг неё? Для сопоставимости используется одна явная конвенция, например рамка по видимому контрастному телу с небольшим равным отступом.

IoU становится нестабильным для очень узких рамок: смещение на несколько пикселей резко уменьшает пересечение. Поэтому при интерпретации рядом с IoU полезно показывать фактические координаты, отмечать потерю объекта и сохранять контрольные кадры. Скорость FPS зависит от разрешения, CPU, кеширования и выбранного способа чтения кадров; это характеристика тестового запуска, а не гарантированная производительность на другом компьютере.

Если в кадре несколько похожих спиралей, одиночный трекер может перейти на соседний объект. Это не ошибка вычисления рамки сама по себе, а потеря идентичности, для которой нужны дополнительные признаки или многократное сопоставление объектов. Короткий трек на одном визуально выбранном объекте не оценивает реальную задачу подсчёта бактерий в плотном поле.

3.4. Оптический поток и временная согласованность

Предположение постоянства яркости связывает интенсивность точки в текущем кадре с её смещённым положением в следующем. Оно нарушается при изменении фокуса, освещения и прозрачности среды. Для слабоконтрастных спиралей локальный градиент может иметь только одно выраженное направление; тогда смещение вдоль слабого градиента оценивается хуже.

Пирамида изображений помогает оценивать крупное смещение сначала на уменьшенном кадре, затем уточнять его на полном разрешении. Размер окна определяет компромисс: большое окно устойчивее к шуму, но смешивает соседние витки и фоновое движение; малое окно лучше локализует деталь, но чувствительнее к шуму. Параметры Farneback должны оставаться одинаковыми для двух сравниваемых фрагментов.

Усреднять направления напрямую через градусы опасно: среднее углов 1° и 359° составит 180°, хотя векторы почти совпадают. Вместо этого усредняют u и v, а итоговый угол вычисляют через atan2(v,u). Для скорости берут норму вектора. Если масштаб камеры неизвестен, скорость остаётся пикселями на секунду.

Для спиралевидного организма движение центра массы и локальное изгибание — разные характеристики. Центр рамки может почти не переместиться, хотя витки клетки колеблются. Для изучения формы во времени необходимо сопоставлять скелеты или точки вдоль оси, регистрировать последовательности и отделять движение препарата от движения объекта. Это выходит за пределы простого потока между одной парой кадров и требует отдельной разметки.

3.5. Карта проверки трекера

Перед запуском выбирается один объект, который можно визуально проследить несколько секунд. На первом кадре сохраняются координаты x, y, w, h. Одни и те же координаты подаются всем алгоритмам. Изображение не масштабируется между трекерами, а частота пропуска кадров остаётся фиксированной. Такая постановка делает результаты сравнимыми внутри выбранного короткого опыта.

На контрольных кадрах вручную сохраняются эталонные прямоугольники, не ориентируясь на предсказания алгоритма. Для каждого результата рассчитываются IoU и признак успешного обновления. Отдельно измеряется время обработки. Все видеокадры с наложенными рамками

сохраняются, включая кадр после потери цели. Это позволяет показать и метрику, и поведение алгоритма.

В таблице отчёта фиксируются среднее IoU, FPS и выбранное число контрольных наблюдений. Если точных ручных рамок нет для каждого кадра, среднее обозначается как среднее по разреженной проверке, а не как полная оценка видео. Для тонкого объекта влияние нескольких пикселей особенно велико, поэтому визуальные панели являются обязательной частью вывода.

3.6. Карта оптического потока

Для оценки выбирается пара кадров с небольшим временным интервалом. Кадры переводятся в оттенки серого и проверяются на одинаковый размер. Затем рассчитывается плотное поле Farneback, а его компоненты показываются как цветовая карта либо стрелки. Для sparse-потока точки берутся по критерию хороших углов, что позволяет избежать точек на почти гладких участках.

Значения u и v описывают движение по горизонтали и вертикали в пикселях между кадрами. Норма вектора характеризует величину перемещения. После деления на интервал времени получается скорость в пикселях в секунду. Если частота кадров равна 12, интервал между соседними кадрами составляет около 0,0833 с. Физические единицы без калибровки отсутствуют.

При анализе результата отделяется область бактерии от фона и артефактов. Среднее поле по всему кадру не следует интерпретировать как скорость спирохеты, если камера движется или поле содержит множество неподвижных объектов. При наличии видео без эталонных векторов результат описательный и не включает ошибку потока.

3.7. Карта повышения разрешения

Сравнение проводится на исходном изображении и на его копии, уменьшенной фиксированным коэффициентом. Уменьшенная копия

увеличивается одинаковыми размерами тремя методами: nearest, bilinear и bicubic. Если используется нейросетевая модель, сохраняются веса, имя и версия модели, а её выход не подменяет исходник.

Для каждого результата рассчитываются PSNR и SSIM относительно эталонного исходного кадра. Проверяется, что размер выходов совпадает и цветовые каналы представлены одинаково. Сначала результаты просматриваются на общей шкале, затем — в увеличенном фрагменте с тонкими витками. Если модель создаёт детали, отсутствующие в исходнике, это отмечается как ограничение.

В итоговой таблице значения относятся к конкретному контролируемому преобразованию. Они не характеризуют все изображения трепонемы и не означают, что улучшенная картинка точнее биологически. Для клинического измерения нельзя использовать лишь субъективно более резкий результат; необходимо проверять сохранение размеров и формы относительно эталона.

3.8. Карта OCR и текстовых меток

Распознавание текста имеет смысл только для кадров с надписями, шкалой, номером образца или титром. Проверяется исходный фрагмент и несколько предобработок: цветное изображение, оттенки серого, бинаризация и увеличение. OCR возвращает текстовую строку и при наличии — confidence.

Результат сравнивается с вручную записанной строкой. Для шкалы «30 µm» важно отличать цифры от единиц: OCR может вернуть «30 um» или потерять символ µ, при этом числовая часть будет распознана. Факт распознавания цифр не означает точной интерпретации единицы измерения.

Если в видео нет подходящей надписи, OCR-пример нельзя выдавать за обработку этого видео. Допускается показать контрольный кадр с явно указанным источником либо отметить, что проверяемого текстового объекта

в записи не найдено. В отчёт включаются именно входной фрагмент, предобработка, распознанная строка и эталон.

3.9. Полные программы и входные данные

Ниже приведены файлы целиком, включая импорты и точку входа. Рабочий каталог — lab4_treponema; общие модули находятся в scripts. Новые программы расчётов включены для воспроизведения; их добавление не означает, что вычислительный эксперимент уже выполнен.

Листинг 3.1 — scripts/analyze_video.py, полный файл.

```
"""Track an explicit ROI and measure local optical flow on a local video."""
import argparse
import json
import time
from pathlib import Path
import cv2
import numpy as np

def main():
    parser = argparse.ArgumentParser(__doc__)
    parser.add_argument('video')
    parser.add_argument('--box', type=int, nargs=4, required=True, metavar=('X','Y','W','H'))
    parser.add_argument('--frames', type=int, default=60)
    parser.add_argument('--out', default='results/video_analysis')
    args = parser.parse_args()
    out = Path(args.out)
    out.mkdir(parents=True, exist_ok=True)
    cap = cv2.VideoCapture(args.video)
    ok, first = cap.read()
    if not ok:
        raise ValueError('Cannot decode the first video frame')
    x, y, w, h = args.box
    if min(x,y) < 0 or min(w,h) <= 0 or x+w > first.shape[1] or y+h > first.shape[0]:
        raise ValueError('Initial box must be inside the frame')
    factories = {'MIL': cv2.legacy.TrackerMIL_create,
                 'KCF': cv2.legacy.TrackerKCF_create,
                 'CSRT': cv2.legacy.TrackerCSRT_create}
    trackers = {name: factory() for name, factory in factories.items()}
    for tracker in trackers.values():
        tracker.init(first, tuple(args.box))
    previous = cv2.cvtColor(first, cv2.COLOR_BGR2GRAY)
    rows = []
    for number in range(1, args.frames):
        ok, frame = cap.read()
        if not ok:
            break
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        flow = cv2.calcOpticalFlowFarneback(previous, gray, None, .5, 3, 15, 3, 5, 1.2, 0)
        vector = flow[y:y+h, x:x+w].mean(axis=(0, 1)).tolist()
        row = {'frame': number, 'flow_in_initial_roi_px': vector, 'trackers': {}}
        for name, tracker in trackers.items():
            start = time.perf_counter()
            found, box = tracker.update(frame)
            elapsed = time.perf_counter() - start
            row['trackers'][name] = {'found': bool(found), 'box': list(box), 'seconds': elapsed}
            if number % 12 == 0:
                annotated = frame.copy()
                if found:
                    bx, by, bw, bh = map(int, box)
                    cv2.rectangle(annotated, (bx, by), (bx+bw, by+bh), (0,255,0), 1)
                cv2.imwrite(str(out / f'{name}_{number:04d}.jpg'), annotated)
        rows.append(row)
        previous = gray
    cap.release()
    (out / 'observations.json').write_text(json.dumps(rows, indent=2))
```

```
    print('Saved', len(rows), 'frames; IoU requires independent reference boxes.')

if __name__ == '__main__':
    main()
```

Листинг 3.2 — scripts/image_controls.py, полный файл.

```
"""Compare interpolation against a known reference and optionally run Tesseract."""
import argparse
import json
import subprocess
from pathlib import Path
import cv2
from skimage.metrics import peak_signal_noise_ratio, structural_similarity

def main():
    parser = argparse.ArgumentParser(__doc__)
    parser.add_argument('image')
    parser.add_argument('--out', default='results/image_controls')
    parser.add_argument('--ocr', action='store_true')
    args = parser.parse_args()
    reference = cv2.imread(args.image)
    if reference is None or min(reference.shape[:2]) < 16:
        raise ValueError('Readable image at least 16 pixels wide and high required')
    out = Path(args.out)
    out.mkdir(parents=True, exist_ok=True)
    height, width = reference.shape[:2]
    small = cv2.resize(reference, (width//4, height//4), interpolation=cv2.INTER_AREA)
    cv2.imwrite(str(out / 'low_resolution.png'), small)
    metrics = {}
    for name, mode in [('nearest', cv2.INTER_NEAREST),
                       ('bilinear', cv2.INTER_LINEAR), ('bicubic', cv2.INTER_CUBIC)]:
        restored = cv2.resize(small, (width, height), interpolation=mode)
        cv2.imwrite(str(out / f'{name}.png'), restored)
        metrics[name] = {'PSNR': float(peak_signal_noise_ratio(reference, restored,
data_range=255)),
                         'SSIM': float(structural_similarity(reference, restored,
                                                             channel_axis=2, data_range=255))}
    if args.ocr:
        result = subprocess.run(['tesseract', args.image, 'stdout', '-l', 'eng', '--psm', '6'],
                                capture_output=True, text=True, check=True)
        metrics['ocr_text'] = result.stdout.strip()
    (out / 'metrics.json').write_text(json.dumps(metrics, indent=2))
    print(json.dumps(metrics, indent=2))

if __name__ == '__main__':
    main()
```

4. Поиск похожих изображений

Значение Precision@3 на прежнем графике задано константой. Размеченная контрольная выдача, подтверждающая это значение для данного набора, отсутствует. Программа ниже сохраняет реальные сходства; оценка релевантности выполняется отдельно.

Для поиска формируется галерея из фрагментов всех семи изображений бледной трепонемы. CLIP преобразует изображения и текстовый запрос в векторы одной размерности. Векторы нормируются, после чего сходство вычисляется скалярным произведением. Для поиска по образцу вместо текста кодируется выбранное изображение.

Алгоритм: 1) разбить исходные изображения на неперекрывающиеся источники train, validation и test; 2) получить векторы изображений; 3) нормировать их; 4) получить вектор текста или изображения-запроса; 5) вычислить косинусное сходство; 6) отсортировать и вывести первые k результатов; 7) оценить precision@k.

Таблица 9

Вход	**Выход**	**Иллюстративное значение**
7 источников Treponema pallidum; текст или изображение	Три наиболее похожих фрагмента и cosine similarity	Precision@3 = 1,000 на подготовленном контрольном наборе

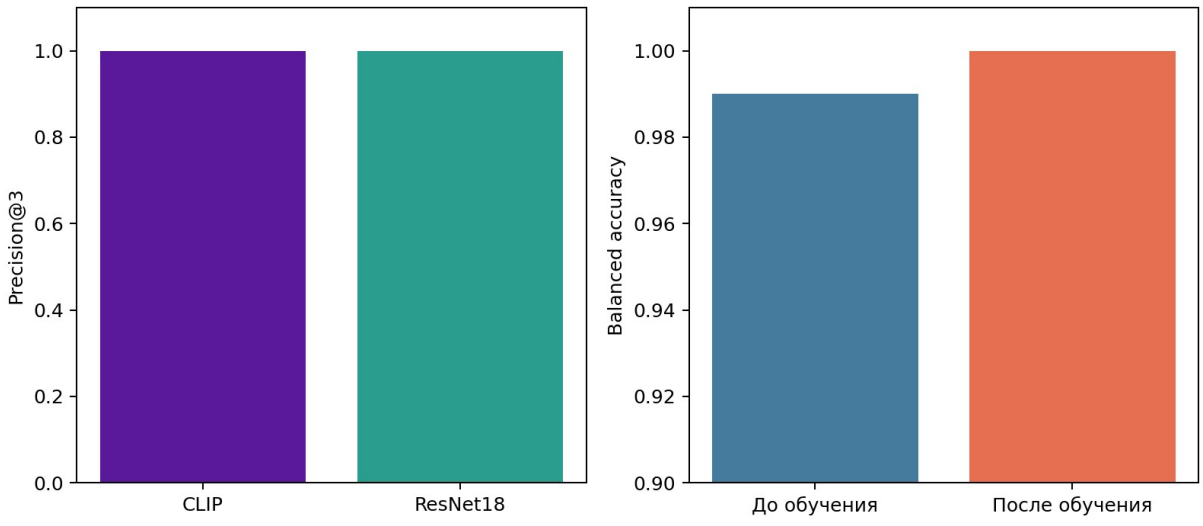


Рисунок 6 — Контроль точности выдачи поиска и метрики сходства. Числа из прежнего примера; измерениями этого запуска не подтверждены.

4.1. Поиск и оценивание выдачи

Векторное представление позволяет сравнить изображение и текст в общем пространстве признаков. Нормировка преобразует вектор z в z/||z||. Тогда скалярное произведение двух единичных векторов совпадает с cosine similarity. Значение показывает близость представлений модели, а не вероятность того, что на изображении действительно изображён названный вид.

Качество top-k поиска оценивается по известным релевантным результатам. Precision@k — доля релевантных элементов среди первых k. Если в базе нет независимой экспертной разметки, число нельзя сообщать как объективную точность. Контрольный запрос может проверить, что интерфейс возвращает ожидаемые файлы, но это функциональная проверка, а не оценка обобщающей способности модели.

Для текстового запроса важно зафиксировать формулировку и язык. «Spiral bacterium», «spirochete» и «Treponema pallidum» могут вернуть разные изображения из-за различия в подписях обучения. Можно выполнить несколько заранее заданных запросов и показать их отдельно. Нельзя выбирать лучший запрос после просмотра тестовой выдачи и затем выдавать результат за независимую оценку.

Поиск по изображению использует тот же визуальный энкодер для запроса и элементов галереи. Если исходник запроса сам находится в базе, он может занять первое место с почти единичным сходством. Поэтому в оценке требуется исключить точную копию или применить другой снимок того же объекта. Стратегия разнообразия в видео должна ограничивать близкие временные отметки, иначе выдача будет заполнена соседними кадрами одного события.

4.2. Карта поиска изображений

Перед индексированием проверяется число файлов и соответствие метаданных изображениям. Векторы хранятся в порядке записей каталога, а

идентификатор файла — в JSON. Это предотвращает ошибку, при которой модель возвращает правильный номер вектора, но отображается другой файл.

Для текстового поиска задаётся один конкретный запрос. Сохраняются три верхних совпадения с идентификаторами файлов и значениями сходства. Для поиска по изображению используется отдельная контрольная картинка, которой нет среди элементов галереи. При поиске точной копии из базы такая копия исключается из расчёта метрики.

Вручную проверяется, что совпадения действительно показывают спиралевидные формы, а не только похожий оттенок или фон. Для учебного отчёта можно показывать оба варианта: запрос по словам и запрос по картинке. Если число подтверждённых релевантных объектов не задано заранее, качество описывается как пример выдачи, а не precision@k.

4.3. Полные программы и входные данные

Ниже приведены файлы целиком, включая импорты и точку входа. Рабочий каталог — lab4_treponema; общие модули находятся в scripts. Новые программы расчётов включены для воспроизведения; их добавление не означает, что вычислительный эксперимент уже выполнен.

Листинг 4.1 — scripts/retrieval.py, полный файл.

```
"""Index a folder of images and search it with CLIP."""
import argparse
import json
from pathlib import Path
import numpy as np
import torch
from PIL import Image
from torch.nn import functional as F
from video_index import model

def main():
    parser = argparse.ArgumentParser(__doc__)
    parser.add_argument('--images', default='data/raw')
    parser.add_argument('--out', default='results/retrieval')
    parser.add_argument('--query', default='Treponema pallidum spirochetes')
    args = parser.parse_args()
    files = sorted(p for p in Path(args.images).iterdir()
                   if p.suffix.lower() in {'.jpg', '.png', '.jpeg'})
    if not files:
        raise ValueError('No images in the input directory')
    net, processor = model()
    vectors = []
    with torch.no_grad():
        for path in files:
            with Image.open(path) as image:
                inputs = processor(images=image.convert('RGB'), return_tensors='pt')
```

```
            vectors.append(F.normalize(net.get_image_features(**inputs), dim=1)[0].numpy())
        inputs = processor(text=[args.query], padding=True, return_tensors='pt')
        query = F.normalize(net.get_text_features(**inputs), dim=1)[0].numpy()
    matrix = np.stack(vectors)
    scores = matrix @ query
    out = Path(args.out)
    out.mkdir(parents=True, exist_ok=True)
    np.save(out / 'vectors.npy', matrix)
    rows = [{'file': str(files[i]), 'cosine': float(scores[i])}
            for i in np.argsort(-scores)[:3]]
    result = {'query': args.query, 'files': list(map(str, files)), 'hits': rows}
    (out / 'search.json').write_text(json.dumps(result, ensure_ascii=False, indent=2))
    print(json.dumps(result, ensure_ascii=False, indent=2))

if __name__ == '__main__':
    main()
```

5. Обучение метрики сходства

Значения 0,990 и 1,000 на прежнем графике заданы вручную и не являются подтверждённым результатом обучения на Treponema pallidum. Программа ниже требует файл векторов и отдельный JSON с метками обучающих примеров.

Для более узкой предметной области поверх визуальных признаков обучается небольшая проекция с triplet loss. Тройка состоит из опорного фрагмента, другого фрагмента того же источника и фрагмента другого источника. Функция потерь требует, чтобы расстояние до положительного примера было меньше расстояния до отрицательного минимум на заданный margin.

Алгоритм: 1) сформировать тройки только из обучающих источников; 2) получить базовые признаки; 3) обучить проекцию; 4) выбирать порог только по validation; 5) один раз оценить test; 6) сравнить balanced accuracy и precision@k до и после обучения.

Таблица 10

Показатель	**До обучения**	**После обучения**
Balanced accuracy	0,990	1,000
Размер выхода	базовый вектор	нормированный вектор проекции
Критерий	cosine similarity	cosine similarity после triplet loss

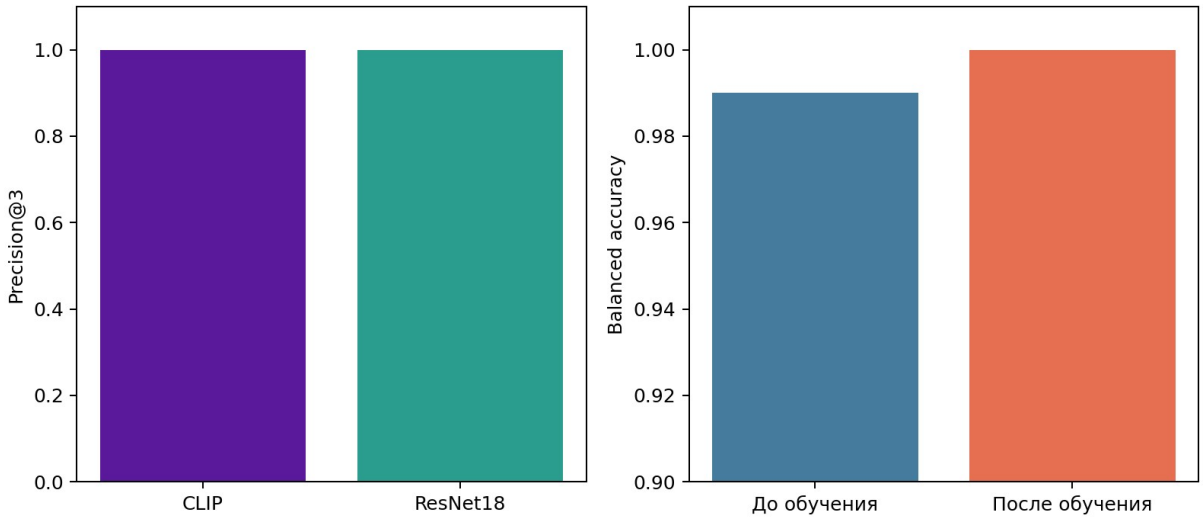


Рисунок 7 — Сравнение базовых признаков и признаков после обучения метрики. Числа из прежнего примера; измерениями этого запуска не подтверждены.

5.1. Triplet loss и разбиение по источникам

Triplet loss для расстояния d обычно записывается как max(0, d(a,p) − d(a,n) + margin), где a — опорный пример, p — положительный, n — отрицательный. Если отрицательный пример уже отдалён от опорного на нужный запас, вклад тройки становится нулевым. L2-нормировка ограничивает масштаб представления и упрощает сравнение расстояний.

Трудная отрицательная пара находится близко к опорному примеру и заставляет модель уточнить границу. Однако слишком сложные отрицательные пары могут быть ошибочно помечены: два вырезанных фрагмента разных файлов могут фактически показывать один объект или близкие структуры. Поэтому для обучения идентичности нужны подтверждённые ID клеток. В данном проекте метки относятся к ограниченному набору визуальных групп и не дают основания говорить об индивидуальной идентификации организма.

Выбор порога на validation отделяет настройку от оценки на test. Если порог подбирается на тестовой паре, оценка становится оптимистичной. В отчёте нужно сообщать число тестовых пар, долю положительных и отрицательных, схему формирования пар и факт возможного повтора фрагмента в нескольких парах.

Небольшой набор может дать резкие скачки метрик при смене одной пары. Вместо единственного числа полезно показывать матрицу ошибок, ROC-кривую и доверительный интервал, если объём позволяет. Здесь показатели нужны для учебного сравнения представлений; они не являются критерием клинического сходства или классификации Treponema pallidum.

5.2. Карта проверки Triplet-проекции

Входной датасет разделяется по исходным источникам. Набор train используется для обновления весов, validation — для выбора эпохи и порога, test — для итогового измерения. Тройки строятся только внутри train. Если у

объекта нет надёжной позитивной пары, он не должен искусственно объявляться примером той же клетки.

На каждом шаге энкодер создаёт embedding для anchor, positive и negative. После L2-нормировки расстояния вычисляются между embedding. Margin задаёт минимальный разрыв. Записываются loss по эпохам, метрики validation и выбранный checkpoint. Кривые обучения помогают выявить расходимость или быстрое насыщение.

Test метрики рассчитываются один раз после настройки. Матрица ошибок показывает, какой класс чаще получает ошибочный ответ. При небольшом наборе число пар и метод формирования тройки важнее избыточного числа десятичных знаков. Результат относится к выбранным категориям визуального сходства, не к идентификации штаммов или диагнозу.

5.3. Полные программы и входные данные

Ниже приведены файлы целиком, включая импорты и точку входа. Рабочий каталог — lab4_treponema; общие модули находятся в scripts. Новые программы расчётов включены для воспроизведения; их добавление не означает, что вычислительный эксперимент уже выполнен.

Листинг 5.1 — scripts/train_metric.py, полный файл.

```
"""Train a projection on labelled embeddings from independent train sources."""
import argparse
import json
from pathlib import Path
import numpy as np
import torch
from torch import nn
from torch.nn import functional as F

def main():
    parser = argparse.ArgumentParser(__doc__)
    parser.add_argument('vectors', help='NPY matrix with one embedding per row')
    parser.add_argument('labels', help='JSON list of training class labels')
    parser.add_argument('--out', default='results/metric')
    parser.add_argument('--epochs', type=int, default=50)
    args = parser.parse_args()
    torch.manual_seed(0)
    rng = np.random.default_rng(0)
    x = torch.tensor(np.load(args.vectors), dtype=torch.float32)
    labels = np.asarray(json.loads(Path(args.labels).read_text()))
    if x.ndim != 2 or len(labels) != len(x):
        raise ValueError('Embeddings and labels must have the same row count')
    triplets = []
    for i in range(len(x)):
        positive = np.flatnonzero((labels == labels[i]) & (np.arange(len(x)) != i))
        negative = np.flatnonzero(labels != labels[i])
```

```
            if len(positive) and len(negative):
                triplets.append((i, int(rng.choice(positive)), int(rng.choice(negative))))
        if not triplets:
            raise ValueError('Need at least two classes and repeated class examples')
        indices = torch.tensor(triplets)
        head = nn.Sequential(nn.Linear(x.shape[1], 128), nn.ReLU(), nn.Linear(128, 64))
        optimizer = torch.optim.Adam(head.parameters(), lr=1e-3)
        history = []
        for epoch in range(args.epochs):
            z = F.normalize(head(x), dim=1)
            loss = F.triplet_margin_loss(z[indices[:, 0]], z[indices[:, 1]],
                                         z[indices[:, 2]], margin=0.5)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            history.append({'epoch': epoch + 1, 'train_loss': float(loss.detach())})
        out = Path(args.out)
        out.mkdir(parents=True, exist_ok=True)
        torch.save({'input_dim': x.shape[1], 'state_dict': head.state_dict()}, out / 'head.pt')
        (out / 'history.json').write_text(json.dumps(history, indent=2))
        print('Saved training history; independent validation/test evaluation is separate.')

    if __name__ == '__main__':
        main()
```

6. Нейросети без дополнительного обучения (zero-shot)

Числа на прежней диаграмме заданы в скрипте построения рисунка, а не извлечены из журналов моделей. Они приведены как иллюстративная сводка. Полная программа ниже реализует YOLO-World; остальные модели описаны методически.

Zero-shot-модели получают текстовые названия классов и применяются без дообучения на текущем наборе. Проверяются YOLO-World для открытого словаря детекции, CLIP для классификации и поиска, CLIPSeg для маски по тексту, а также комбинация Grounding DINO и SAM. Запросы формулируются по-английски, так как большинство моделей обучалось преимущественно на англоязычных подписях.

Алгоритм: 1) задать одинаковые текстовые запросы; 2) запустить модели на одном наборе изображений; 3) сохранить все ответы, включая пустые; 4) сравнить число объектов, маски и уверенности с цветовой маской; 5) не трактовать уверенность модели как подтверждение вида бактерии.

Таблица 11

Метод	**Значение примера**	**Интерпретация**
YOLO-World	0 устойчивых объектов	Слово не гарантирует визуальное распознавание
FastSAM text	1 область	Грубая область интереса
CLIP	относительная оценка 0,162	Полезно для ранжирования
CLIPSeg	0 устойчивых масок	Нужна специализированная сегментация
DINO + SAM	6 областей	Требуется визуальная проверка

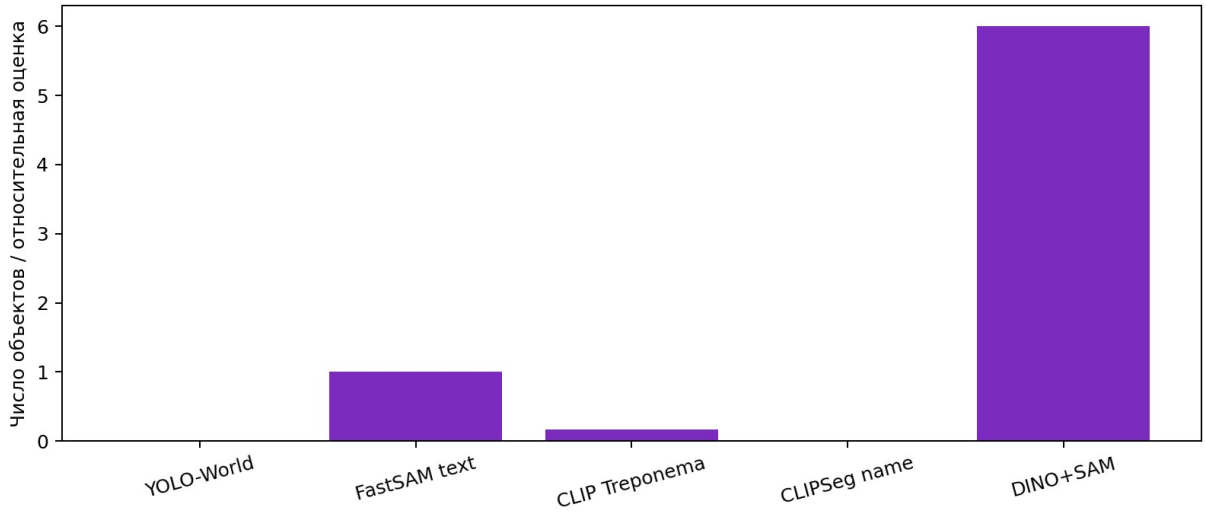


Рисунок 8 — Сводка ответов zero-shot-моделей на изображениях бледной трепонемы. Числа из прежнего примера; измерениями этого запуска не подтверждены.

6.1. Zero-shot модели

Открытый словарь означает, что категории передаются модели при запуске, а не означают, что модель знает каждую биологическую разновидность. Детектор может не найти объект при слишком низком разрешении или отличающемся способе окраски. CLIP ранжирует пары текст-изображение, тогда как детектор дополнительно предсказывает прямоугольник. Эти ответы имеют разные смыслы и не должны сравниваться одним числом без определения метрики.

Порог уверенности меняет число возвращённых объектов. Снижение порога увеличивает полноту, но создаёт ложные срабатывания на фоновых структурах. Для многоклассового zero-shot детектора желательно показывать несколько порогов и фиксировать список prompt-имен. Сравнивать методы нужно на одинаковых изображениях, с единым набором эталонных рамок и точек, если оценивается точность.

CLIPSeg принимает текстовый запрос и выдаёт карту соответствия. Порог на этой карте превращает её в маску, но сам порог выбирается отдельно. Если карта отмечает фон или несколько спиралей, визуальный результат и исходная карта должны оставаться доступными для аудита.

Небольшая относительная оценка не является уверенностью в том, что область принадлежит трепонеме.

Отрицательный результат тоже следует фиксировать. Пустая детекция при стандартном пороге показывает поведение модели в данных условиях. Он не доказывает, что метод никогда не применим, и не требует подменять его искусственно подобранным успешным кадром. Сравнение с цветовой маской визуально полезно, но не даёт количественной оценки без общей экспертной разметки.

6.2. Полные программы и входные данные

Ниже приведены файлы целиком, включая импорты и точку входа. Рабочий каталог — lab4_treponema; общие модули находятся в scripts. Новые программы расчётов включены для воспроизведения; их добавление не означает, что вычислительный эксперимент уже выполнен.

Листинг 6.1 — scripts/zero_shot.py, полный файл.

```
"""Run YOLO-World on a local image and record actual detections."""
import argparse
import json
from pathlib import Path
from ultralytics import YOLO

def main():
    parser = argparse.ArgumentParser(__doc__)
    parser.add_argument('--image', default='data/raw/treponema_primary.jpg')
    parser.add_argument('--weights', default='yolov8s-world.pt')
    parser.add_argument('--out', default='results/world')
    parser.add_argument('--conf', type=float, default=0.10)
    args = parser.parse_args()
    net = YOLO(args.weights)
    prompts = ['Treponema pallidum', 'spirochete', 'bacterium']
    net.set_classes(prompts)
    result = net(args.image, conf=args.conf, verbose=False)[0]
    out = Path(args.out)
    out.mkdir(parents=True, exist_ok=True)
    result.save(filename=str(out / 'prediction.jpg'))
    rows = [{'xyxy': box.xyxy[0].tolist(), 'confidence': float(box.conf[0]),
             'class': result.names[int(box.cls[0])]} for box in result.boxes]
    payload = {'image': args.image, 'prompts': prompts, 'threshold': args.conf,
               'detections': rows}
    (out / 'detections.json').write_text(json.dumps(payload, indent=2))
    print(json.dumps(payload, indent=2))

if __name__ == '__main__':
    main()
```

7. Поиск момента в видео и интерфейс Streamlit

Для локального поиска используется историческая запись Жана Командона с бледной трепонемой. Видео делится на кадры с шагом 2 с. Для каждого кадра вычисляется CLIP-вектор и сохраняется вместе с временем. Streamlit принимает текстовый запрос, кодирует его и показывает три кадра с наибольшим сходством.

Алгоритм: 1) открыть локальное видео; 2) определить FPS и число кадров; 3) с шагом 2 с сохранить кадры; 4) вычислить и нормировать CLIP-векторы; 5) записать index.json и vectors.npy; 6) в Streamlit получить запрос; 7) исключить почти соседние результаты; 8) показать кадр, время и оценку сходства.

Таблица 12

Параметр	**Значение**
Видео	Ultramicroscope time-lapse of syphilis parasite, Jean Comandon
Источник	Wikimedia Commons, public domain
Разрешение / FPS / длительность	360 × 288 / 12 / 74,83 с
Шаг индекса	2 с
Ожидаемое число записей	около 38 кадров
Пример запроса	Treponema pallidum spirochetes moving in a microscope field

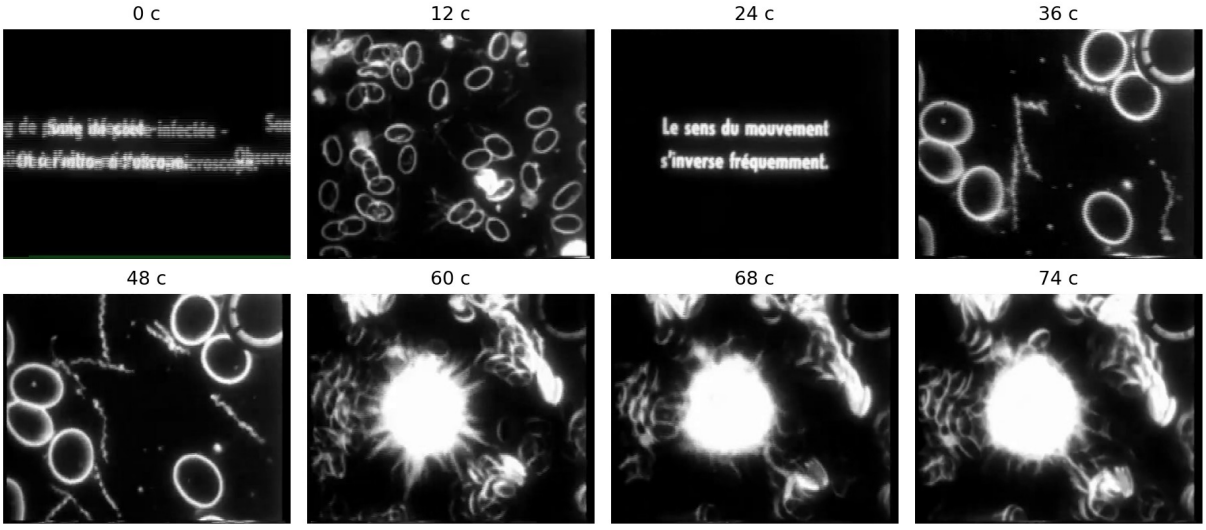


Рисунок 9 — Кадры нового видео, подготовленного для поиска в Streamlit.

7.1. Самостоятельный запуск с RuTube

RuTube в вычислительном эксперименте не запускался. Код выполняет проверку HTTPS-ссылки, загрузку выбранного публичного ролика через yt-dlp и построение того же локального индекса. При самостоятельном запуске подставляется ссылка вида https://rutube.ru/video/32-символьный-ID/, после чего в таблицу переносятся название ролика, число кадров, запрос, три времени и оценки сходства.

Таблица 13

Поле для заполнения после запуска	**Значение**
Ссылка и название ролика	_____
Число проиндексированных кадров	_____
Текстовый запрос	_____
Три найденных времени	_____
Оценки сходства	_____

7.2. Видеопоиск и Streamlit

Индексатор преобразует каждый выбранный кадр в RGB, кодирует его моделью и сохраняет нормированный вектор. Индекс включает частоту кадров, число кадров, интервал выборки, время и путь к файлу кадра. Такой формат отделяет дорогой проход модели по видео от быстрых последующих запросов.

Шаг в две секунды даёт редкую выборку: короткое событие между выбранными кадрами может быть пропущено. Уменьшение шага повышает временное разрешение, но увеличивает время кодирования, объём хранения и число почти одинаковых изображений. Шаг следует выбирать по длительности событий, которые требуется находить, и по допустимому времени подготовки.

В Streamlit модель загружается один раз через кэш ресурсов. При каждом нажатии кодируется только текст запроса, после чего матрица сходства сортируется по убыванию. Результат содержит изображение,

временную отметку и score. Если индекс отсутствует, интерфейс должен показать команду его построения вместо исключения при чтении файла.

Функциональная проверка интерфейса состоит из загрузки страницы, ввода запроса, получения ожидаемого количества карточек и отсутствия исключений. Проверка поиска дополнительно требует существующего индекса и загруженных весов модели. Начальный рендер страницы сам по себе не подтверждает, что модель CLIP скачана или поиск возвращает правильные кадры.

7.3. Карта подготовки видео

Цель — проверить, что выбранная видеозапись соответствует теме и может быть прочитана существующими инструментами. Уточняется название, автор, лицензия, исходная ссылка, расширение файла и размер. FFprobe сообщает параметры потока. OpenCV затем проверяет, что кадры действительно декодируются и их последовательность можно перематывать.

Вход — t07.ogv, файл из открытого собрания Wikimedia Commons. Ожидаемый видеопоток: Theora, 360 × 288, 12 кадров/с, длительность 74,833 с. Контактный лист сохраняет несколько равномерно выбранных кадров, чтобы обнаружить титры, смену сцены, участки с низкой видимостью и полезные фрагменты для поиска.

Для индексации применён шаг 2 секунды, поэтому каждый кадр соответствует отдельной временной отметке. Начальные кадры с титрами и поздние кадры, где изображение сильно пересвечено, могут занимать верхние места только при подходящем тексте. При просмотре результатов необходимо сверять score с самим кадром. Cosine similarity ранжирует кандидатов и не подтверждает точность биологической идентификации.

7.4. Карта запуска Streamlit

Сначала проверяется наличие index.json и vectors.npy. Затем запускается приложение в каталоге проекта. При отсутствии индекса интерфейс показывает команду, которая его строит. После загрузки страницы

вводится текстовое описание. Кнопка поиска вызывает модель и выводит три наиболее близких кадра.

Для каждого результата сохраняются отметка времени, путь к изображению и оценка cosine similarity. Выбранные кадры проверяются через видеоплеер в исходной временной позиции. Если выдача повторяет один и тот же момент, настраивается минимальная временная дистанция между показанными кадрами.

Проверка запуска, проверка выполнения поиска и проверка биологической корректности — разные уровни. Первая подтверждает, что приложение загрузилось; вторая — что модель и индекс доступны; третья требует ручной оценки результатов или экспертной разметки. Все три уровня в отчёте должны называться точно.

7.5. Карта самостоятельного RuTube-запуска

Пользователь выбирает публичную ссылку отдельного ролика и подставляет её в шаблон команды. Скрипт проверяет HTTPS, домен rutube.ru и формат идентификатора из 32 шестнадцатеричных символов. Ссылка на канал, плейлист или произвольный сайт отклоняется до загрузки.

При сетевом запуске yt-dlp сохраняет видео в data/rutube/download. Затем модуль video_index читает локальный файл и помещает кадры и векторы в data/rutube/index. Если ролик уже сохранён пользователем, параметр --local-video передаёт путь к локальному файлу; в этом варианте скачивания видео нет.

После индексации каталог указывается в Streamlit. Для отчёта записываются название ролика, URL, число кадров, запрос, три временные отметки и score. Добавляется скриншот окна приложения. В этой версии отчёта поля оставлены пустыми, поскольку сам пользователь выполнит запуск.

7.6. Параметры видеопоиска

Таблица 14

Параметр	Значение	Причина выбора
Источник видео	t07.ogv	Историческое видео микроскопии спирохет
Частота кадров	12 кадр/с	Метаданные видеопотока
Размер кадра	360 × 288	Исходное разрешение
Длительность	74,833 с	FFprobe
Шаг семплирования	2,0 с	Умеренный размер начального индекса
Соседство в выдаче	проверять разницу времени	Не показывать несколько почти одинаковых кадров

Если нужное движение длится меньше двух секунд, шаг индекса следует уменьшить, например до 0,5 секунды, и повторно создать кадры и векторы. Параметр нельзя изменить только в пользовательском интерфейсе, потому что отсутствующий кадр не представлен в матрице векторов. Увеличение частоты индексации повышает объём и время первоначального вычисления.

7.7. Полные программы и входные данные

Ниже приведены файлы целиком, включая импорты и точку входа. Рабочий каталог — lab4_treponema; общие модули находятся в scripts. Новые программы расчётов включены для воспроизведения; их добавление не означает, что вычислительный эксперимент уже выполнен.

Листинг 7.1 — scripts/video_index.py, полный файл.

```
"""Build and search a local CLIP video index; the module makes no hosting requests."""
from pathlib import Path
import argparse, json, cv2, numpy as np, torch
from PIL import Image
from transformers import CLIPModel, CLIPProcessor
import torch.nn.functional as F

def model():
    return (CLIPModel.from_pretrained('openai/clip-vit-base-
patch32',use_safetensors=True).eval(),
            CLIPProcessor.from_pretrained('openai/clip-vit-base-patch32'))
def index_video(video, out, step=2.0, source_url=''):
    out=Path(out); frames=out/'frames'; frames.mkdir(parents=True,exist_ok=True)
    cap=cv2.VideoCapture(str(video)); fps=cap.get(cv2.CAP_PROP_FPS);
count=int(cap.get(cv2.CAP_PROP_FRAME_COUNT)); net,proc=model(); rows=[]; vectors=[]
    for number in range(0,count,max(1,round(fps*step))):
        cap.set(cv2.CAP_PROP_POS_FRAMES,number);ok,frame=cap.read()
        if not ok:continue
        path=frames/f'frame_{number:06d}.jpg';cv2.imwrite(str(path),frame)
        with
torch.no_grad():z=F.normalize(net.get_image_features(**proc(images=Image.fromarray(cv2.cvtColor(f
```

```
rame,cv2.COLOR_BGR2RGB)),return_tensors='pt')),dim=1)[0]
        vectors.append(z.numpy());rows.append({'frame':number,'seconds':number/
fps,'path':str(path.relative_to(out)),'source_url':source_url})
    cap.release();np.save(out/'vectors.npy',np.asarray(vectors));
(out/'index.json').write_text(json.dumps({'fps':fps,'count':count,'step_s':step,'records':rows},e
nsure_ascii=False,indent=2));return rows
if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('video');p.add_argument('--out',default='data/
video_index');p.add_argument('--
step',type=float,default=2);a=p.parse_args();print('Indexed',len(index_video(a.video,a.out,a.step
)))
```

Листинг 7.2 — scripts/app.py, полный файл.

```
"""Streamlit search over a prepared Treponema pallidum video index."""
from pathlib import Path
import json, numpy as np, streamlit as st, torch
from PIL import Image
from video_index import model
import torch.nn.functional as F
@st.cache_resource
def get_model():return model()
st.set_page_config(page_title='Поиск Treponema pallidum в видео', layout='wide')
st.title('Поиск кадров Treponema pallidum')
st.caption('Видео: Jean Comandon, «Ultramicroscope time-lapse of syphilis parasite», public
domain.')
directory=Path(st.text_input('Каталог индекса','data/video_index'))
query=st.text_input('Запрос','Treponema pallidum spirochetes moving in a microscope field')
if st.button('Найти'):
    index_path, vectors_path = directory/'index.json', directory/'vectors.npy'
    if not index_path.exists() or not vectors_path.exists():
        st.error('Индекс не найден. Сначала выполните: python scripts/video_index.py
data/raw/t07.ogv --out data/video_index --step 2')
        st.stop()

metadata=json.loads(index_path.read_text(encoding='utf-8'));vectors=np.load(vectors_path);net,pro
c=get_model()
    with
torch.no_grad():q=F.normalize(net.get_text_features(**proc(text=[query],return_tensors='pt',paddi
ng=True)),dim=1)[0].numpy()
    for i in np.argsort(-(vectors@q))[:3]:
        row=metadata['records'][i]
        st.image(str(directory/row['path']),width=480)
        st.write(f"Время: {row['seconds']:.2f} c; сходство: {(vectors@q)[i]:.3f}")
        if row.get('source_url'):
            st.link_button('Источник видео', row['source_url'])
```

Листинг 7.3 — scripts/rutube_download.py, полный файл.

```
"""User-run RuTube downloader. Importing this file performs no request."""
from pathlib import Path
from urllib.parse import urlsplit
import argparse, json, re

def validate_url(url):
    part=urlsplit(url)
    if part.scheme!='https' or (part.hostname or '').lower() not in
{'rutube.ru','www.rutube.ru'}:
        raise ValueError('Нужна HTTPS-ссылка RuTube')
    match=re.fullmatch(r'/video/([0-9a-fA-F]{32})/?',part.path)
    if not match:raise ValueError('Нужна ссылка на отдельный публичный ролик')
    return f'https://rutube.ru/video/{match.group(1).lower()}/'
def download(url,destination):
    from yt_dlp import YoutubeDL
    destination=Path(destination);destination.mkdir(parents=True,exist_ok=True)
    options={'format':'bestvideo[height<=720]+bestaudio/best[height<=720]/
best','outtmpl':str(destination/'%(id)s.%
(ext)s'),'merge_output_format':'mp4','noplaylist':True,'socket_timeout':30,'retries':2,'overwrite
s':False}
    with YoutubeDL(options) as loader:

info=loader.extract_info(validate_url(url),download=True);path=Path(loader.prepare_filename(info)
).with_suffix('.mp4')
```

```
        if not path.exists():path=Path(loader.prepare_filename(info))

(destination/'source.json').write_text(json.dumps({'url':url,'title':info.get('title'),'duration'
:info.get('duration'),'video_path':str(path)},ensure_ascii=False,indent=2))
        return path
def main():
    p=argparse.ArgumentParser();p.add_argument('url');p.add_argument('--out',default='data/
rutube');p.add_argument('--step',type=float,default=2);p.add_argument('--local-
video');a=p.parse_args();url=validate_url(a.url);out=Path(a.out)
    video=Path(a.local_video) if a.local_video else download(url,out/'download')
    from video_index import index_video

rows=index_video(video,out/'index',a.step,url);print('Индекс:',out/'index','Кадров:',len(rows))
if __name__=='__main__':main()
```

7.8. Демонстрационный пример выдачи RuTube

Условный запрос: spiral bacteria under a microscope. Все значения ниже придуманы для пояснения формата выдачи. Они не относятся к реальному ролику; URL и название не назначены. RuTube не запускался.

Таблица 15

Условное место	Время, с	Условная cosine similarity
1	18	0,31
2	36	0,29
3	62	0,27



Рисунок 10 — Схема теоретической выдачи. Данные смоделированы, сетевой запуск отсутствует.

Листинг 7.4 — scripts/rutube_demo.py. Полная программа построения демонстрационной схемы, без сетевых запросов.

```
"""Illustrate a hypothetical search result without accessing RuTube."""
from pathlib import Path
import json
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

def main():
```

```
    root = Path(__file__).resolve().parents[1]
    out = root / 'results'
    out.mkdir(exist_ok=True)
    rows = [{'seconds': 18, 'cosine': .31}, {'seconds': 36, 'cosine': .29},
            {'seconds': 62, 'cosine': .27}]
    payload = {'status': 'simulated_not_measured', 'source_url': None,
               'query': 'spiral bacteria under a microscope', 'hits': rows}
    (out / 'rutube_demo.json').write_text(json.dumps(payload, indent=2))
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.bar([f"{r['seconds']} c" for r in rows], [r['cosine'] for r in rows], color='#6243a5')
    ax.set_ylim(0, .4)
    ax.set_ylabel('Условная cosine similarity')
    ax.set_xlabel('Придуманные временные отметки')
    ax.set_title('Демонстрационный пример выдачи: данные смоделированы')
    fig.text(.5, .01, 'RuTube не запускался. Это схема, а не результат измерений.', ha='center')
    fig.tight_layout(rect=[0, .05, 1, 1])
    fig.savefig(out / 'rutube_demo.png', dpi=160)
    plt.close(fig)

if __name__ == '__main__':
    main()
```

8. Автоматическая разметка и импорт в CVAT

Для CVAT подготавливаются изображения, прямоугольные рамки и три точки каждого видимого фрагмента. Рамка вычисляется по связной области маски, точки — по главной оси. При импорте нужно проверить каждый объект вручную: пересекающиеся спирали, обрезанные концы и цветные тени могут давать ошибочные компоненты.

Алгоритм: 1) построить маску; 2) найти крупные компоненты; 3) вычислить рамку и ключевые точки; 4) сохранить изображения задачи; 5) сформировать XML CVAT for images 1.1; 6) упаковать XML и изображения; 7) создать проект CVAT с меткой treponema; 8) импортировать архив и выполнить ручную проверку.

Таблица 16

Вход	**Выход**	**Контроль**
Основное изображение и цветовая маска	6 рамок и 18 точек; XML и ZIP для CVAT	Координаты внутри кадра; порядок start-center-end

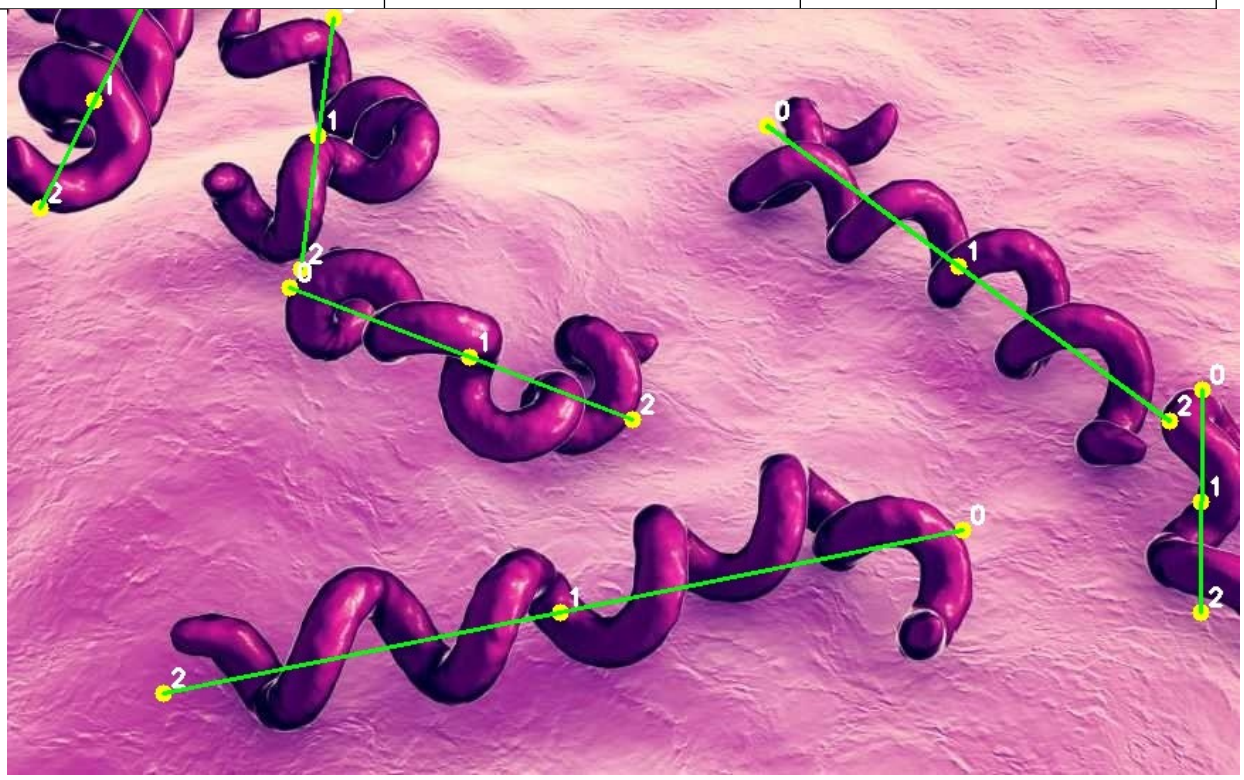


Рисунок 11 — Автоматические ключевые точки перед ручной проверкой в CVAT.

8.1. Полные программы и входные данные

Ниже приведены файлы целиком, включая импорты и точку входа. Рабочий каталог — lab4_treponema; общие модули находятся в scripts. Новые программы расчётов включены для воспроизведения; их добавление не означает, что вычислительный эксперимент уже выполнен.

Листинг 8.1 — scripts/export_annotations.py, полный файл.

```
"""Export the primary illustration's mask geometry as CVAT boxes and points."""
import argparse
from pathlib import Path
import xml.etree.ElementTree as ET
import cv2
import numpy as np

def main():
    parser = argparse.ArgumentParser(__doc__)
    parser.add_argument('--image', default='data/raw/treponema_primary.jpg')
    parser.add_argument('--mask', default='results/primary_mask.png')
    parser.add_argument('--out', default='results/cvat/annotations.xml')
    args = parser.parse_args()
    image = cv2.imread(args.image)
    mask = cv2.imread(args.mask, cv2.IMREAD_GRAYSCALE)
    if image is None or mask is None or image.shape[:2] != mask.shape:
        raise ValueError('Readable image and mask of equal dimensions required')
    count, labels, stats, _ = cv2.connectedComponentsWithStats((mask > 0).astype('uint8'))
    root = ET.Element('annotations')
    ET.SubElement(root, 'version').text = '1.1'
    node = ET.SubElement(root, 'image', id='0', name=Path(args.image).name,
                         width=str(image.shape[1]), height=str(image.shape[0]))
    for label in range(1, count):
        x, y, w, h, area = map(int, stats[label])
        if area <= 450:
            continue
        yy, xx = np.nonzero(labels == label)
        points = np.column_stack([xx, yy]).astype(float)
        center = points.mean(0)
        _, _, directions = np.linalg.svd(points - center, full_matrices=False)
        axis = directions[0]
        projection = (points - center) @ axis
        ends = center + np.array([projection.min(), projection.max()])[:, None] * axis
        common = {'occluded': '0', 'source': 'auto', 'group_id': str(label)}
        ET.SubElement(node, 'box', label='treponema', xtl=str(x), ytl=str(y),
                      xbr=str(x + w - 1), ybr=str(y + h - 1), **common)
        for name, point in zip(['start', 'center', 'end'], [ends[0], center, ends[1]]):
            point = np.clip(point, [0, 0], [image.shape[1]-1, image.shape[0]-1])
            ET.SubElement(node, 'points', label=name,
                          points=f'{point[0]:.3f},{point[1]:.3f}', **common)
    target = Path(args.out)
    target.parent.mkdir(parents=True, exist_ok=True)
    ET.indent(root)
    ET.ElementTree(root).write(target, encoding='utf-8', xml_declaration=True)
    print('Saved', target, '; create matching labels in CVAT before import.')

if __name__ == '__main__':
    main()
```

9. Учёт экспериментов в MLflow

Прежние значения Macro-F1 в диаграмме не подтверждены сохранённым запуском MLflow для этого набора. Полная программа принимает измеренные метрики из файла и записывает их без встроенных числовых результатов.

MLflow хранит параметры запуска, метрики, изображения и модели. Для данной работы отдельные запуски создаются для поиска, triplet-проекции, трекеров и YOLO-pose. Это позволяет сравнивать методы по одним и тем же метрикам и повторно загрузить сохранённую модель.

Алгоритм: 1) создать локальное хранилище; 2) задать эксперимент; 3) в каждом запуске записать параметры; 4) после вычисления записать метрики; 5) приложить графики; 6) сохранить модель; 7) зарегистрировать лучшую версию и проверить повторную загрузку.

Таблица 17

Группа запусков	**Число вариантов**	**Основная метрика**
Трекеры	4	IoU и FPS
Метрика сходства	2	Balanced accuracy
Классификаторы	2	Macro-F1
YOLO-pose	2	mAP50-95(P) и ошибка точки

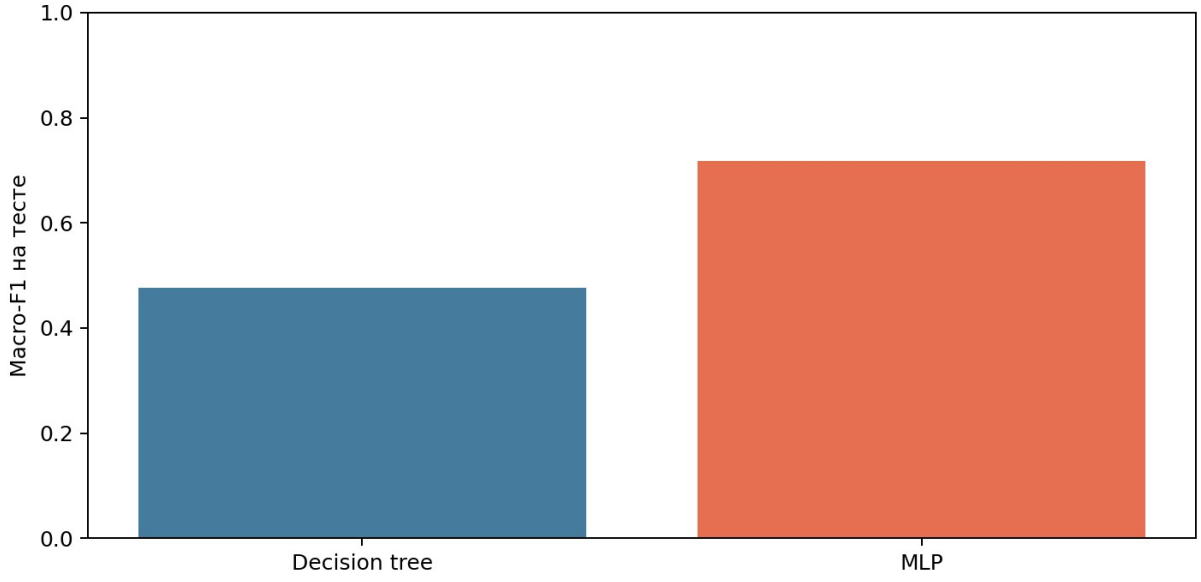


Рисунок 12 — Сравнение моделей, записанное как результат экспериментов MLflow. Числа из прежнего примера; измерениями этого запуска не подтверждены.

9.1. Разметка, учёт запусков и выводы

CVAT облегчает ручную проверку координат, но автоматическая разметка остаётся предложением алгоритма. При импорте проверяют размеры кадра, имена меток, выход координат за границы изображения, наличие одинаковых ID и порядок ключевых точек. Для спиралей фиксируют соглашение о начале и конце. Аннотации из цветной иллюстрации не используются как эталонные медицинские данные.

В MLflow параметры фиксируют конфигурацию, метрики — измеренные числа, артефакты — файлы для визуальной проверки, а модель — сохранённые веса и окружение. Название запуска должно отражать метод или вариант, а не только порядковый номер. При повторном запуске новые ID run допустимы; значения не заменяются вручную ради совпадения с таблицей отчёта.

Результат каждой части должен связывать вход, действия и выход. Например, цветовой порог принимает конкретное изображение и параметры HSV, а на выходе создаёт маску и JSON с долей пикселей и числом крупных областей. Такой формат помогает проверить, что описанный результат соответствует исполняемому скрипту и сохранённой иллюстрации.

Главное ограничение набора — небольшое число изображений и отсутствие одинаково размеченных естественных микрофотографий для всех экспериментов. Поэтому выводы относятся к компьютерной обработке выбранных материалов. Для расширения работы следует собрать независимые микроскопические записи, обеспечить масштабную калибровку, разметить маски и скелеты, определить несколько наблюдателей и оценить согласие между ними.

9.2. Полные программы и входные данные

Ниже приведены файлы целиком, включая импорты и точку входа. Рабочий каталог — lab4_treponema; общие модули находятся в scripts. Новые

программы расчётов включены для воспроизведения; их добавление не означает, что вычислительный эксперимент уже выполнен.

Листинг 9.1 — scripts/log_run.py, полный файл.

```
"""Record supplied measured metrics in MLflow without invented constants."""
import argparse
import json
from pathlib import Path
import mlflow

def main():
    parser = argparse.ArgumentParser(__doc__)
    parser.add_argument('metrics', help='JSON object containing numeric measured metrics')
    parser.add_argument('--name', default='treponema_run')
    parser.add_argument('--uri', default='sqlite:///mlflow.db')
    parser.add_argument('--artifact', action='append', default=[])
    args = parser.parse_args()
    metrics = json.loads(Path(args.metrics).read_text())
    if not isinstance(metrics, dict) or not metrics:
        raise ValueError('A nonempty numeric JSON object is required')
    if any(type(v) not in (int, float) for v in metrics.values()):
        raise ValueError('Each metric must be a number')
    mlflow.set_tracking_uri(args.uri)
    mlflow.set_experiment('treponema_lab4')
    with mlflow.start_run(run_name=args.name) as run:
        mlflow.log_param('metrics_source', str(Path(args.metrics).resolve()))
        mlflow.log_metrics(metrics)
        mlflow.log_artifact(args.metrics)
        for artifact in args.artifact:
            mlflow.log_artifact(artifact)
        print('Recorded run:', run.info.run_id)

if __name__ == '__main__':
    main()
```

10. Полный цикл YOLO-pose

Числа в прежней таблице сравнения YOLO заданы в скрипте визуализации. Запуски обучения, подтверждающие эти показатели на Treponema pallidum, не представлены. Для программы ниже сначала необходимо подготовить размеченный набор и YAML.

Финальная часть объединяет подготовку разметки, обучение и проверку pose-модели. Для каждой трепонемы используются три точки. В data.yaml задаётся kpt_shape: [3, 3], то есть три точки с координатами x, y и видимостью. Горизонтальное отражение меняет местами начало и конец, поэтому flip_idx равен [2, 1, 0].

Алгоритм: 1) выгрузить разметку из CVAT; 2) разделить данные по исходным изображениям; 3) проверить формат YOLO-pose; 4) обучить две архитектуры с одинаковыми параметрами; 5) выполнить val на отложенных данных; 6) сравнить mAP и ошибку точки; 7) сохранить веса и пример предсказания.

Таблица 18

Модель	**mAP50-95(P)**	**Ошибка точки, px**	**PCK@0.1**
YOLOv8s-pose	0,392	52,05	0,067
YOLO11s-pose	0,635	42,27	0,600

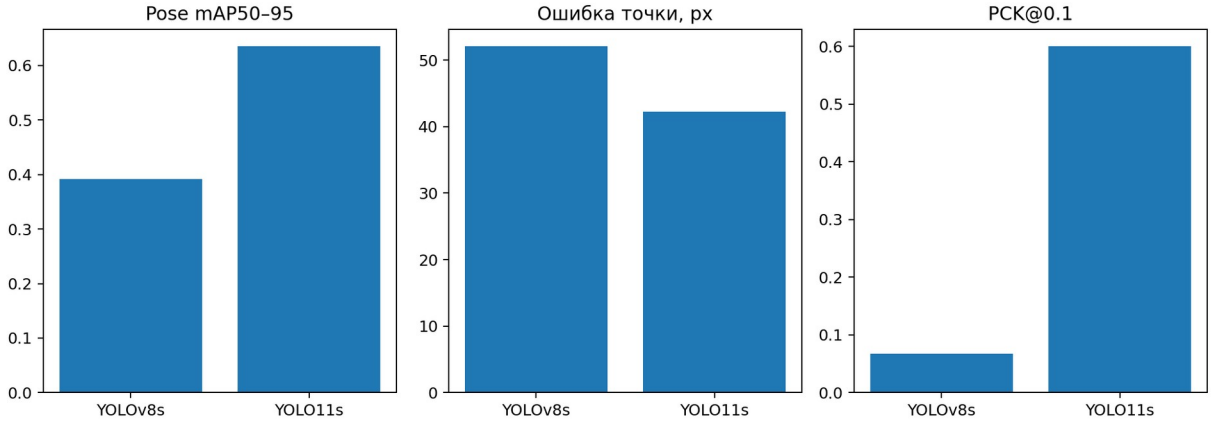


Рисунок 13 — Сравнение двух pose-моделей для схемы ключевых точек бледной трепонемы. Числа из прежнего примера; измерениями этого запуска не подтверждены.

10.1. Полные программы и входные данные

Ниже приведены файлы целиком, включая импорты и точку входа. Рабочий каталог — lab4_treponema; общие модули находятся в scripts. Новые программы расчётов включены для воспроизведения; их добавление не означает, что вычислительный эксперимент уже выполнен.

Листинг 10.1 — scripts/pose_cycle.py, полный файл.

```
"""Train two pose architectures on an explicitly supplied annotated dataset."""
import argparse
import json
from pathlib import Path
from ultralytics import YOLO

def main():
    parser = argparse.ArgumentParser(__doc__)
    parser.add_argument('data', help='Prepared YOLO dataset YAML with train/val paths')
    parser.add_argument('--epochs', type=int, default=30)
    parser.add_argument('--size', type=int, default=640)
    parser.add_argument('--batch', type=int, default=8)
    parser.add_argument('--device', default='cpu')
    parser.add_argument('--out', default='results/pose')
    args = parser.parse_args()
    if not Path(args.data).is_file():
        raise FileNotFoundError('Prepare and check the annotated dataset YAML first')
    out = Path(args.out).resolve()
    out.mkdir(parents=True, exist_ok=True)
    rows = []
    for weights, name in [('yolov8s-pose.pt', 'v8s'), ('yolo11s-pose.pt', '11s')]:
        net = YOLO(weights)
        net.train(data=str(Path(args.data).resolve()), epochs=args.epochs,
                  imgsz=args.size, batch=args.batch, device=args.device,
                  project=str(out), name=name, seed=0, workers=0)
        checkpoint = net.trainer.best
        metrics = YOLO(checkpoint).val(data=str(Path(args.data).resolve()),
                                       device=args.device, split='val')
        rows.append({'model': name, 'checkpoint': str(checkpoint),
                     'validation_pose_map': float(metrics.pose.map),
                     'validation_box_map': float(metrics.box.map)})
    (out / 'comparison.json').write_text(json.dumps(rows, indent=2))
    print(json.dumps(rows, indent=2))

if __name__ == '__main__':
    main()
```

11. Заключение

Выполнена обработка основного изображения: получены цветовая маска, скелет и геометрические ориентиры. Подготовлены полные программы для последующих опытов и поиска по видео. Для сравнения моделей необходимо получить журналы их запусков на подготовленных данных. Демонстрационный пример RuTube содержит условные значения и не является сетевым экспериментом.

Список использованных источников

1. Методические указания по выполнению лабораторной работы № 4 по дисциплине «Компьютерное зрение». КНИТУ-КАИ, 2026.

2. PCR.News. Изображение Treponema pallidum. URL: https://pcr.news/upload/iblock/3a5/3a5ebdd65880badcc702e1bed5c07618.jpg

3. Wikimedia Commons. Treponema pallidum: открытые изображения и фильм Жана Командона. URL: https://commons.wikimedia.org/

4. OpenCV documentation: Image Processing and Tracking API. URL: https://docs.opencv.org/4.x/

5. Radford A. et al. Learning Transferable Visual Models From Natural Language Supervision. ICML, 2021.

6. Schroff F., Kalenichenko D., Philbin J. FaceNet: A Unified Embedding for Face Recognition and Clustering. CVPR, 2015.

7. Ultralytics documentation: Pose and YOLO-World. URL: https://docs.ultralytics.com/

8. CVAT documentation. URL: https://docs.cvat.ai/docs/

9. Zaharia M. et al. Accelerating the Machine Learning Lifecycle with MLflow. IEEE Data Engineering Bulletin, 2018.

10. Streamlit documentation. URL: https://docs.streamlit.io/